

Ministry of Higher Education and Scientific Research

PFE Code: IM40

**University of Manouba
Higher Institute of Multimedia Arts**



End of studies project

Submitted to obtain The Bachelor's Degree in Computer Science

Cloud Reservation and Billing system of cloud resources

Created by:

Zaineb Arfaoui & Mohamed Mensi

Supervised by:

Dr. Amine Kechiche - ISAMM

Mr. Badis Saadi, Mr. Nawfel Hamrouni, Mr. Jamel Aloui

El Khawarizmi Computing Center (CCK)

Academic Year: 2025/2026

ACKNOWLEDGEMENT

First and foremost, we want to thank God the Almighty, who gave us the strength, the health, intelligence, perseverance and patience to do this work in the right conditions.

*We express our deepest gratitude to our academic director **Mrs. Saoussen Krichen**, as well as our academic supervisors **Mr. Badis Saadi**, **Mr. Naoufel Hamrouni** and **Mr. Jamel Aloui**, for their valuable guidance, continuous support, patience, and insightful suggestions throughout this project.*

*A special thanks goes to our mentor at the institute, **Mr. Amine Kechiche** for the quality of his tutoring, his trust, his patience, his relevant remarks, and his encouragement.*

We also want to express our gratitude to all the professors at the Higher Institute of Multimedia Arts of Manouba (ISAMM) for their efforts and for the quality of their teaching and training they have provided us with during these past two years.

We are equally grateful to our friends who have been close to us with their encouragement, criticism, and advice and to anyone who has contributed to the success of this project.

Finally, our sincere thanks to the members of the jury for the honor of evaluating our work.

إهداء

قال تعالى:

وَقُلْ اَعْمَلُوا فَسَيَرَى اللَّهُ عَمَلَكُمْ وَرَسُولُهُ وَالْمُؤْمِنُونَ

[سورة التوبة: ١٠٥]

إلى من غرسوا في قلبي حب العلم والعمل،

إلى من كانوا الدعامة الأولى في طريقي،

إلى والديّ العزيزين: والدي الحبيب **لسعد** ووالدي العزيزة **بسمة**،

شكرًا لدعائكما، لصبركما، ولقلبيكما الذي احتواني في كل الظروف.

إلى نبض روحي وسندي الأقرب،

إلى أختي العزيزة ندى، التي كانت لي أمًا ثانية وصديقة مخصصة،

أهديك ثمرة هذا الجهد بكل حبّ وامتنان.

وإلى أخي الحبيب محمد أمين،

رفيق الدرب، ومصدر التشجيع والدعم المستمر،

شكرًا لوجودك في كل المراحل.

إلى أصدقائي الذين كانوا نورًا في لحظات التعب،

وشركاء الفرح في لحظات الإنجاز.

وإلى رفيقات سكني، من حملن أيامي بحضورهن وابتساماتهن،

لكم جميعًا، أهدي هذا العمل بكل فخر وامتنان.

زينب العرفاوي



إهداء

قال تعالى:

****وَأَنْ لَّيْسَ لِلْإِنْسَانِ إِلَّا مَا سَعَى وَأَنَّ سَعْيَهُ سَوْفَ يُرَى****
[سورة النجم: ٣٩-٤٠]

إلى والدتي الحبيبة كوثر التي روت ظمئي للعلم منذ الصغر بحبها الغامر اللامحدود، وغرست في قلبي بذور المعرفة بحنانها الدافئ.

إلى والدي العزيز نسيم الذي كان الملاذ الآمن وسط عواصف الحياة، يمدني بالهدوء والسكينة حين تشتد بي المصاعب.

إلى أختي مرام رفيقة دربي ومؤازرة أحلامي، التي آمنت بطموحاتي قبل أن أومن بها أنا، وكانت أول من مد لي يد الدعم والتشجيع.

إلى أخي فراس المحرك النابض للحياة في عائلتنا، الذي ينثر البهجة والحيوية أينما حل، ويكسر رتابة الأيام بروحه المرح.

إلى جميع أصدقائي في كل محطات حياتي، الذين رافقوني في رحلة النمو والتطور، وأضاءوا طريقي بتشجيعهم المستمر ووقوفهم إلى جانبي.

إلى كامل أفراد عائلتي الكريمة الذين كانوا دعامي الراسخة، وركيزتي الثابتة، ومصدر إلهامي وطاقتي في جميع مراحل حياتي، والقيمة الثابتة التي لا تتزعزع مهما تقلبت الأحوال.

إلى روح جدتي الغالية فاطمة التي نحتت شخصيتي بأناملها الحانية، وغرست فينا أجمل القيم وأنبهنا، واهتمت بأدق تفاصيل حياتنا مهما بدت صغيرة، تاركة في قلوبنا أثراً عميقاً لا يمحوه الزمن. إلى روح جدي الحبيب عز الدين منارة الحكمة ومرجع العائلة في أوقات الشدة.

إليكُم جميعاً أهدي هذا العمل، تقديراً لكل ما قدمتموه لي من حب ودعم وإيمان.

محمد المانسي

Contents

List of Figures	vi
List of Tables	viii
General Introduction	1
1 GENERAL PROJECT PRESENTATION	3
1.1 The host organization	4
1.1.1 Presentation of the host organization	4
1.1.2 CCK Main Missions	4
1.1.3 CCK Services	4
1.2 Project overview	5
1.2.1 Scope	5
1.2.2 Problem statement	5
1.3 Study of existing systems	6
1.3.1 CCK existing system analysis	6
1.3.2 Description of existing systems in the market	7
1.3.3 Solution overview	11
1.4 Adopted Methodology	12
1.4.1 Project Management Methodology	12
1.4.2 Modeling language	16
Conclusion	16
2 Sprint 0 : Analysis and Specification of Requirements	17
2.1 Requirements Analysis	18
2.1.1 Identification of actors	18
2.1.2 Identification of functional requirements	19
2.1.3 Non functional requirements	20
2.2 Requirements specification	21
2.2.1 Global Use Case Diagram	21
2.2.2 Class diagram	22
2.3 Project management with SCRUM	24

2.3.1	Product backlog	24
2.3.2	Sprint and release planning	25
2.4	Global architecture of the project	26
2.4.1	Logic Architecture	26
2.4.2	Physical architecture	30
2.5	Work environment	31
2.5.1	Hardware environment	31
2.5.2	Software environment	31
	Conclusion	36
3	Release 1	37
3.1	Requirements Analysis and Specification	38
3.1.1	Release 1 Backlog	38
3.1.2	Use Case Diagram of Release 1	43
3.1.3	Analysis	44
3.2	Design	46
3.2.1	Sequence Diagrams	46
3.2.2	Class Diagram of release 1	52
3.3	Realization	53
	Conclusion	56
4	Release 2	57
4.1	Requirements Analysis and Specification	58
4.1.1	Release 2 Backlog	58
4.1.2	Use Case Diagram of Release 2	61
4.1.3	Analysis	62
4.2	Design	63
4.2.1	Sequence Diagrams	63
4.2.2	Class Diagram of release 2	65
4.3	Realization	65
	Conclusion	68
	Conclusion and perspectives	69
	Bibliography	70

List of Figures

1.1	CCK's logo	5
1.2	CCK's organizational chart	5
1.3	Azure logo	7
1.4	Storage Metrics Comparison for 1-Year and 3-Year Reserved Capacity in Azure	8
1.5	IBM logo	9
1.6	Empiricism pillars	13
1.7	Scrum Process	15
1.8	UML logo	16
2.1	Global Use Case Diagram.	21
2.2	Class diagram.	23
2.3	Sprints planning	25
2.4	3-tier Architecture	27
2.5	Application Architecture	27
2.6	Design Pattern MVC	28
2.7	Design Pattern MVVM	29
2.8	Deployment Diagram	30
2.9	Drawio Logo.	31
2.10	Overleaf Logo.	32
2.11	Postman Logo.	32
2.12	MySQL Logo	32
2.13	Angular Logo.	32
2.14	Flask Logo	33
2.15	TypeScript Logo	33
2.16	Visual Studio Code Logo	34
2.17	BootStrap Logo	34
2.18	HTML Logo	34
2.19	CSS Logo	34
2.20	JavaScript Logo	35
2.21	Spring Boot Logo	35

2.22 Figma Logo	36
3.1 Use case diagram of Release 1	44
3.2 Professor Registration Sequence Diagram.	47
3.3 Professor Authentication Sequence Diagram	48
3.4 Admin Authentication Sequence Diagram.	49
3.5 Reservation approach Sequence Diagram	50
3.6 Additional Resources Sequence Diagram.	51
3.7 Available Resources Sequence Diagram.	52
3.8 Class Diagram of release 1	53
3.9 Interface: Professor Registration	53
3.10 Interface: Professor Login	53
3.11 Interface: University Login	54
3.12 Interface: CCK Login	54
3.13 Interface: Resource Reservation	54
3.14 Interface: Manage Profile for Professor	55
3.15 Interface: University Manage Reservations	55
3.16 Interface: University Manage Professors	55
3.17 Interface: CCK Manage Reservations	56
4.1 Use case diagram of Release 2	62
4.2 Generate bills Sequence Diagram	63
4.3 Consult bills Sequence Diagram	64
4.4 Class Diagram of Release 2	65
4.5 Interface: Professor Reports	66
4.6 Interface: Professor Reservations	66
4.7 Interface: Professor Metrics	66
4.8 Interface: University Reports	67
4.9 Interface: University Billing	67
4.10 Interface: University Bill details	67
4.11 Interface: CCK Reports	68
4.12 Interface: CCK Billing	68

List of Tables

1.1	Comparison table.	11
2.1	Table of actors.	18
2.2	Professor Features	19
2.3	University Features	19
2.4	CCK Features	20
2.5	CCK Infrastructure Management Features	20
2.6	Product Backlog	25
2.7	Hardware Environment – Shared Development VM	31
3.1	Release 1 Backlog	38
4.1	Release 2 Backlog	58

LIST OF ACRONYMS

AI:	Artificial Intelligence
API:	Application Programming Interface
CCK:	Centre de Calcul El Khawarizmi
CPU:	Central Processing Unit
ECS:	Elastic Cloud Service
EER:	European Entrepreneurial Region
HTTP:	Hypertext Transfer Protocol
IAM:	Identity and Access Management
JSON:	JavaScript Object Notation
OS:	Operating System
PaaS:	Platform as a Service
RAM:	Random Access Memory
REST API:	Representational State Transfer Application Programming Interface
SQL:	Structured Query Language
UI:	User Interface
VM:	Virtual Machine

General Introduction

The evolution of cloud usage is increasing academically. Universities and institutes are digitizing their data in order to improve the use of digital information technology in the academic environment as well as for the benefit of professors and students. However, this evolution can lead to several challenges in the distribution of resources and the clarity of the information, making it difficult to reserve cloud resources and manage the billing. Also, the problem of manually making reservations that can be laborious, slow, and prone to mistakes.

The goal of this undertaking project, “Cloud Resource Reservation and Billing System,” is to create a web platform that streamlines the entire lifecycle of cloud resource management within academic institutions. This system connects three key stakeholder groups:

- **Professors:** Users who need to reserve cloud resources(virtual machines; storage, RAM, and CPU) for teaching.
- **Universities:** administrators who manage academic staff and need transparency into resource usage. They manage professors and reservations, and they consult university bills.
- **CCK:** administrators that control the cloud resources reservations and universities, set quotas, and manage billing, which is detailed invoices for cost allocation to a university.

This report is therefore divided into 4 chapters:

- **The first chapter :** named ”General Project Presentation” presents the host company, the context of the project, a critical study of the related works, as well as the problem to be solved. Finally, we will explain the methodology adopted to design the project.

- **The second chapter** concerns planning and architecture: In this chapter, we will define the project management methodology, determine the functional requirements, non-functional requirements, and product backlog, plan the releases, and finally give the overall use case diagram as well as the class diagram.
- **The third chapter** presents in detail the achievement of Release 1, with an analysis.
- **The fourth chapter** presents the achievement of Release 2 with a detailed analysis.

In conclusion, the prospects for improving the vision and development of this platform will be discussed at the end of this work.

Chapter 1

GENERAL PROJECT PRESENTATION

Summary

1.1	The host organization	4
1.1.1	Presentation of the host organization	4
1.1.2	CCK Main Missions	4
1.1.3	CCK Services	4
1.2	Project overview	5
1.2.1	Scope	5
1.2.2	Problem statement	5
1.3	Study of existing systems	6
1.3.1	CCK existing system analysis	6
1.3.2	Description of existing systems in the market	7
1.3.3	Solution overview	11
1.4	Adopted Methodology	12
1.4.1	Project Management Methodology	12
1.4.2	Modeling language	16
	Conclusion	16

Introduction

In this chapter, we will present the host company, CCK (Centre de Calcul EL KHAWARIZMI), the general context of our project, and the goals to be achieved. Next, we will address the problem, followed by a study of existing solutions and similar solutions. Then, we will outline the methodology that will be adopted throughout the project.

1.1 The host organization

1.1.1 Presentation of the host organization

The CCK was established in October 1976 after the endorsement of computer science as a university specialty at the Faculty of Sciences of Tunis. At the time, it was providing computing services to institutions, agencies, and administrations of the Ministries of National Education and Higher Education.

The center's mission is to encourage academic, scientific, and technical activity by making available educational courses, sources of scientific and technical information, and administrative applications intended for students and researchers.

1.1.2 CCK Main Missions

- Ensure, organize, promote, and encourage the use of digital technologies in academic and scientific environments.
- Conduct research to enhance the use of digital information technology in academia.
- Support teachers and students by improving digital technology adoption.

1.1.3 CCK Services

- Management of the National University Network
- Hosting of national applications and websites of EER
- Management and monitoring of RNU network security
- RNU messaging / Office 365 DriveCCK and SMS management
- Training, digital production, and technological monitoring

The following figure shows the CCK logo:



Figure 1.1: CCK's logo

The following FIGURE illustrates the CCK organizational chart:

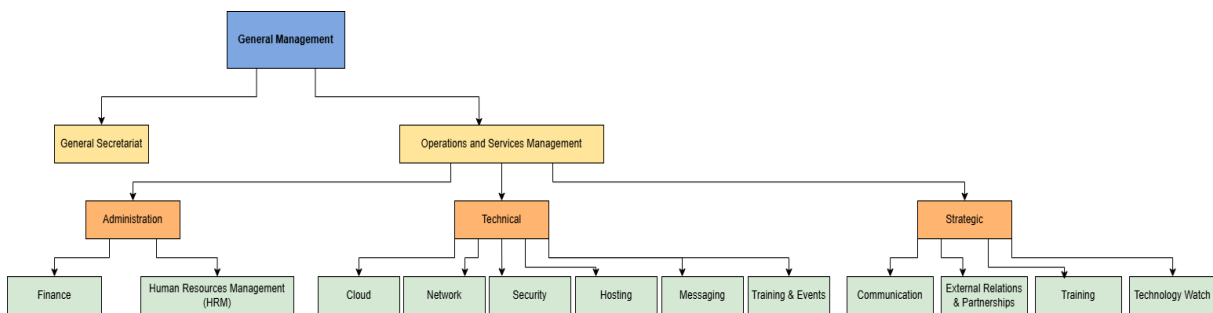


Figure 1.2: CCK's organizational chart

1.2 Project overview

1.2.1 Scope

Our proposed project is entitled "Cloud Resource Reservation and Billing System". It has been proposed as a graduation project for the bachelor's degree in Computer Science from the Higher Institute of Multimedia Arts of Manouba (ISAMM). This internship took place from February 1, 2025, to June 1, 2025 at the CCK (EL KHAWARIZMI Computer Center).

1.2.2 Problem statement

Cloud solutions are changing education by offering a scalable, flexible and inexpensive IT solution. In particular, cloud platforms are very useful in universities to deliver digital learning opportunities. A key challenge within the university infrastructure is managing and delivering computing resources like virtual machines (VM).

Without a reservation system, resources are not allocated fairly, reservations are disorganized, and there is no proper control over who uses what.

The benefit of a cloud-based reservation system is that it allows departments, professors,

and researchers to request, reserve, and allocate resources based on actual ensuring availability and better planning.

Cloud platforms allow fine-grained tracking of who uses which resource, for how long, and at what cost. This process prevents misuse, helps professors allocate resources based on quotas, and allows real-time monitoring of CPU, memory, and storage usage.

Attempting to track cloud consumption without a formal billing system is difficult and can lead to errors, overusage, and abuse.

So, universities need a billing system that does two things: automatically calculates costs based on actual usage and generates clear invoices. And by embedding billing logic into the resource reservation system, universities remain cost-effective while offering flexibility to users.

The purpose of our initiative is to develop a web-based application for universities to easily reserve, track, and manage their cloud resources while also providing an accurate and automated billing and invoicing system.

1.3 Study of existing systems

Existing system analysis is a necessary first step in developing any project; it involves studying the current systems that are related to the project. This enables us to understand the problems, needs, and opportunities for improvement of the current systems.

The suggested remedy and our project implementation process will all be detailed in this part.

1.3.1 CCK existing system analysis

In CCK's current cloud resource management process, all processes are mainly manual without automation. When professors need cloud resources, they must contact the university administrator to submit a request; after, they have to wait a long time, sometimes several days, before the availability is confirmed and the resources are allocated.

Although the CCK cloud infrastructure is hosted on a Huawei-based cloud solution, the system does not allow universities or users access to a few critical features, including access to many user-level features that limit efficiency and transparency.

The major limitations of the current system include:

- There is no academic dashboard for universities to consult resource usage, billing details, or activity statistics.
- Professors don't have an interface to directly request or reserve cloud resources.
- There is no automated resource creation; instead, the allocation requires manual approval from CCK administrators.

- Lack of visual interface for universities to track cloud consumption in real time.
- Lack of billing module.

Due to these limitations with the existing system, it was important to understand how current cloud providers deal with similar problems. Major platforms such as Huawei Cloud, Microsoft Azure, and IBM Cloud offer many advanced resource management and billing systems that provide inspiration for our proposed solution. These providers also offer several pricing models and services used to manage computing resources efficiently.

Before starting our project, we conducted a study of the different solutions available on the market, from which we will present two cloud providers, Azure and IBM.

1.3.2 Description of existing systems in the market

A/ Azure Cloud Services:

Azure Cloud Services is a full cloud platform that offers scalable business solutions with flexible pricing and a wide range of services.



Figure 1.3: Azure logo

✓ Azure Pricing Models and Metrics

Azure offers several pricing structures based on different usage needs:

- 1/ **Pay-As-You-Go:** is a cloud payment method where users are charged only for the resources they actually use. The main metrics are:
 - **Virtual Machines (VMs) metrics:** Usage duration, operating system (OS), VM size and type, storage costs, data transfer.
 - **Storage metrics:** Data transfer, Storage class/Tier(Standard, Cold, Archive), Read/Write operations, Replication and redundancy, Storage type, Storage capacity, Snapshot and Backup storage.
 - **SQL Databases metrics:** Compute resources, Storage, Backup and Retention, Read/Write transactions, Replication, Data transfer.
 - **Function Service metrics:** Invocation count, Memory allocation, Data transfer, Execution time, CPU allocation, Storage.

- **App services(Paas) metrics:** Database usage, Data transfer, Storage, Execution time, Instance scaling, Compute instance.
- **AI services metrics:** Cognitive services, Metrics advisor, Document intelligence, OpenAI service, Speech services, Cost management.

2/ **Reserved Instances:** Discounts of up to 40% for 1-year reservations and 72% for 3-year reservations, requiring upfront or monthly payments. The main metrics are:

- **Virtual Machines (VMs) metrics:** Commitment term, VM specifications, Instance flexibility, Payment option, Reservation scope, Software costs.
- **SQL Databases metrics:** Service type, vCores/Throughput, Reservation term, Region.
- **App services(Paas) metrics:** Compute resources, Reservation duration, Pricing tier, Region, Number of instances.
- **Function Service metrics:** Execution time, Number of executions on top of the reserved capacity.
- **Storage metrics:** Azure storage services offer different access tiers (Hot, Cool, and Archive) with 1-year and 3-year reserved capacity options to optimize costs.
Hot:Frequently accessed
Cool:Backup and disaster recovery
Archive: Historical archives.

This visualization illustrates how committing to 1-year or 3-year reserved storage capacity impacts cost savings and resource utilization.

Azure Storage Reserved Capacity	1-year reserved			3-year reserved		
	Hot	Cool	Archive	Hot	Cool	Archive
100 TB/month	\$1,545	\$840	\$183	\$1,244	\$676	\$168
1 PB/month	\$15,050	\$8,179	\$1,783	\$11,963	\$6,502	\$1,636

Figure 1.4: Storage Metrics Comparison for 1-Year and 3-Year Reserved Capacity in Azure

3/ **Spot Instances:** Eligible only for VMs, it is up to 90% cheaper than Pay-As-You-Go, but if there is an increase in the demand from other users, Azure can reclaim these Spot VMs at any time with little notice. The main metrics are:

- **Virtual Machines (VMs) metrics:** Eviction rate, cost fluctuation based on demand.

4/ **Savings Plans:** can offer you up to 65% off compared to a pay-as-you-go plan. The exact discount depends on factors such as the services you use, the region where they're available, and your commitment to using them regularly for one to three years. The main metrics are:

- **Virtual Machines (VMs), App Services, Azure Functions, SQL Databases metrics:** Committed hourly usage, VM sizes, regions, and service types.

B/ IBM Cloud Services:

A cloud platform designed for businesses in strict industries, offering strong performance, high security, and full compliance. It offers diverse pricing and service models.



Figure 1.5: IBM logo

✓IBM Pricing Models and Metrics

IBM follows similar pricing strategies:

1/ **Pay-As-You-Go:** The main metrics are:

- **Analytics and AI Services metrics:** Processing time, API call volume, model accuracy rate.
- **Storage metrics:** Data transfer, Storage class/Tier(Standard, Cold, Archive), Read/Write operations, Replication and redundancy, Storage type, Storage capacity, Snapshot and Backup storage.
- **SQL Databases metrics:** Compute resources, Storage, Backup and Retention, Read/Write transactions, Replication, Data transfer.

2/ **Reserved Instances:** The main metrics are:

- **Virtual Machines (VMs) metrics:** Commitment term, VM specifications, Instance flexibility, Payment option, Reservation scope, Software costs.

3/ **Spot Instances:** Eligible only for VMs. The main metrics are:

- **Virtual Machines (VMs) metrics:** Eviction rate, cost fluctuation based on demand.

4/ **Savings Plans and Subscriptions:** IBM also provides Savings Plans and Subscription models, which allow fixed pricing for:

- **Networking:** Bandwidth Usage, Latency, Packet Loss Rate(Percentage of lost data packets).
- **Security and identity:** Number of security incidents detected,Authentication Requests,Encryption Strength.
- **Machine Learning and AI:** Processing Time, API Call Volume, Accuracy Rate.
- **Storage:** Data Read/Write Speed, Storage Utilization, Access Frequency.
- **SQL Databases:** Query Latency, Transaction Count, Replication Lag.

Azure and IBM Cloud are robust platforms known for their rich service offerings, scalability, and flexible pricing models. Azure provides a broad range of tools, including advanced AI services and cost-saving options like reserved and spot instances. IBM Cloud focuses on high security, performance, and compliance. Both platforms support hybrid billing models, allowing businesses to optimize costs while maintaining flexibility for changing resource demands.

The following table provides a detailed comparison of key criteria of Azure and IBM Cloud.

Table 1.1: Comparison table.

Criteria	Azure	IBM
Billing Model	Pay-As-You-Go / Reserved Instances / Spot / Savings Plan	Pay-As-You-Go / Reserved Instances / Spot / Savings Plan / Subscription
Services offered and metrics	VMs (Usage duration, OS, Size, Storage, Data Transfer) Storage (Data Transfer, Read/Write, Replication, Access Tier) SQL Databases (Compute, Storage, Backup, Transactions, Replication) Function Services (Invocation count, Execution time, CPU, Memory) App Services (PaaS) (Instance scaling, Compute, Execution time) AI Services (Cognitive services, OpenAI, Speech services) Services	VMs (Usage duration, OS, Size, Storage, Data Transfer) Storage (Data Transfer, Read/Write, Replication, Access Tier) SQL Databases (Query Latency, Transaction Count, Replication Lag) Networking (Bandwidth, Latency, Packet Loss Rate) Security (Security incidents detected, Authentication requests, Encryption Strength) Machine Learning & AI (Processing time, API Calls, Accuracy Rate)
Use cases	Enterprise solution / Serverless applications	Data security / Hybrid solutions
Security & Compliance	ISO certifications / HIPAA / SOC / GDPR	DORA, RGPD, CCPA, SOX, PCI, HIPAA
Availability of services	60 + regions	6 main regions
Ease of integration	Integration with windows and DevOps tools.	Solutions adapted for large enterprises and hybrid cloud.

1.3.3 Solution overview

We propose a cloud resource reservation and billing platform inspired by Microsoft Azure and IBM Cloud providers. Our solution adopts a hybrid billing model that combines reserved instances (for predictable usage) and pay-as-you-go (for additional, on-demand resource requests).

In this hybrid model, the universities receive a monthly invoice according to their monthly reservation, and charges for additional virtual machine (VM) requests are billed separately using a pay-as-you-go model. This ensures flexibility, scalability, and cost transparency.

Our solution is based on four main parts, which we will explain in the following outline:

- **Automated cloud resource reservations:** In the system, a user should be able to request resources; if there are not enough resources available in the system, then the user can request additional resources or wait till the availability of resources or cancel the reservation to change his request.
- **Real-time monitoring of consumption:** The system should provide alerts when the consumption exceeds a specified percentage of the total quota.
- **Automated billing/invoicing:** The platform should generate invoices that detail the consumption of resources to accurately account for billing and documentation.

Through these aspects, the platform improves the process of cloud resource planning and management in educational institutions. It also improves academic institutions' use of cloud resources through the notification system of alerts and reservations while improving the visibility and efficiency of their cloud spend.

1.4 Adopted Methodology

To organize the project effectively and ensure it is delivered on time, it is required to employ an appropriate development approach. A methodology is important for a project because it provides a structured form, ensuring that all essential processes are completed logically and consistently. It also minimizes errors in the development process.

1.4.1 Project Management Methodology

In our project, we will use the agile implementation framework of SCRUM. Scrum is an incremental and iterative agile approach used to manage product development.

Scrum follows a schedule based on time, called sprints, which are short development cycles. Each sprint ends with an increment of a working product. [Sac16]

Next, we will summarize some benefits of using SCRUM:

- **Flexibility to change:** SCRUM adapts to requirement changes during the project, improving responsiveness to customer needs.
- **Continuous improvement:** Through the regular sprint review, teams constantly improve processes for product quality.
- **Reduced risk:** By working in an incremental way, Scrum reduces project risks and puts efforts on delivering high-value functionality early .

1/ SCRUM Principles:

Agile Scrum is based on three primary pillars: transparency, inspection, and adaptation [Sac16]:

- **Transparency** : sharing any critical information for the members and stakeholders to have a clear vision of the project's progress.
- **Inspection** : It allows the team to check the work and processes regularly to identify any project risks.
- **Adaptation** : It means adjusting the work or approach based on feedback to ensure the team remains aligned with project goals despite changing conditions.

The following figure illustrates the three pillars of empiricism that form the foundation of the Scrum framework.

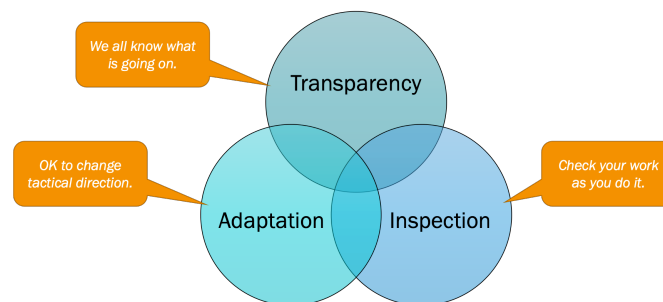


Figure 1.6: Empiricism pillars

2/ Scrum Actors:

The main actors of Scrum are essential for understanding its implementation. [CS16]

- **Product owner** : represents customers and/or users by transcribing their needs and defining and prioritizing product requests; he must have functional business expertise.
- **Development team** : it manages itself autonomously and is responsible for product development. Its size must remain small (7 to 9 people). Self-organized, devoid of hierarchy, and without specialized roles, it is "made up of people who contribute to producing a result in each sprint, the value creators".
- **Scrum master** : a member of the development team, he is also the team leader. He is not the project manager, but he must facilitate the implementation of Scrum. He strives to ensure the team works under good conditions and focuses on the project's objective, paying attention to ensuring the various phases of Scrum are followed.

3/ Scrum Terminology:

In Scrum we use the following terminology [CS16] :

- **Daily Scrum** : A sharing moment, a short "ceremony" or meeting (approximately 15 minutes) held each day with team members and the product owner to update everyone on their activities, determine the day's tasks, and identify obstacles preventing the team from achieving the sprint goal.
- **Product Backlog**: A product backlog is a list of product features that have been prioritized. In essence, it is a shared to-do list of all the items to be addressed. *Retrospective*: A retrospective is a "ceremony" at the end of each sprint where the Scrum Master, product owner, and other team members discuss those things that the team can do better. It is an important moment in the scrum process, as it represents one of the essential principles of the agile manifesto: at regular intervals, the team reflects upon how to improve in the next sprint and changes behavior.
- **Sprint Backlog** : A list of features to be completed during a sprint. The corresponding task list is established during sprint planning. The task load is determined by the developers.
- **Sprint Planning** : This involves the team and the product owner determining the goal and content of the upcoming sprint. User stories are broken down into tasks, each of which is subject to a workload estimate, expressed in hours or days, by the team. The main result is the sprint backlog.
- **Sprint Review** : This essentially consists, at the end of each cycle, of publicly demonstrating the results of the sprint, collecting feedback from stakeholders, and making decisions.
- **User Story** :backlog item. A system requirement to be developed is a brief description of a feature as seen by the user (customer).

In the document, we decided to combine two sprints into a single release. We did this to avoid duplication and to better organize the report while not sacrificing the integrity of the Scrum process. Both sprints were carried out and reviewed separately, but they are both reported together as one release because they are functionally related and to also enhance clarity in the final report.

- **Release**: In Scrum, a release is a planned delivery cycle that pulls together high-priority product backlog items to deliver business value within a clearly defined scope, timeline, and budget. It clearly stated what would be delivered, highlighted the major risks, and communicated the best information to stakeholders about when and how the work would be completed. [Sac16]

4/ SCRUM Process:

The Scrum process can be summarized by the FIGURE below.

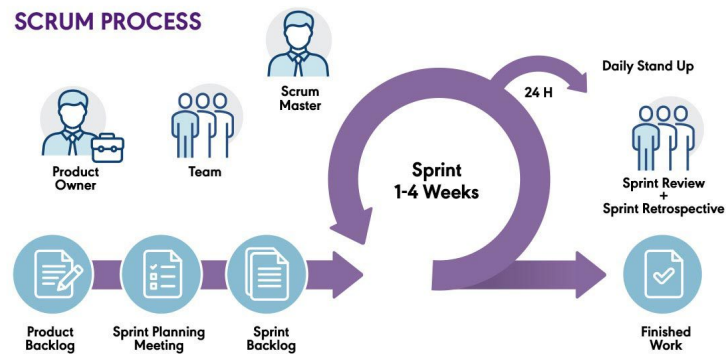


Figure 1.7: Scrum Process

Examining FIGURE 1.7 shows that:

- Product Backlog: The Product Owner creates and maintains the Product Backlog.
- Sprint Planning Meeting: A meeting is held to plan which Product Backlog items will be selected for the Sprint.
- Sprint Backlog: After planning, the selected tasks are moved into the Sprint Backlog.
- Sprint (1–4 Weeks): Development happens over a fixed duration Sprint (1 to 4 weeks).
- Daily Stand-up (every 24 hours): Daily meeting during the sprint to track progress.
- Sprint Review + Sprint Retrospective: At the end of the sprint
 - **Sprint Review:** Demonstrate the finished work.
 - **Sprint Retrospective:** Reflect on how the team worked and how to improve.
- Finished Work: Deliver a completed increment of the product.

1.4.2 Modeling language

For modeling our project, we will be using UML.

The Unified Modeling Language (UML) is a graphical way of modeling the structure and behavior of complex computer software or systems. UML was developed as a straightforward and standardized way to model and understand computer applications in an easy way by visually modeling the system. UML diagrams provide a way to visualize, process, construct, and describe software systems contextually and simply [MG00].



Figure 1.8: UML logo

Conclusion

In this chapter, we placed the project in its general context to establish the necessary reference points. We started with a description of the overall framework of the project work and the hosting organization. Next, we provided a description of the existing system before presenting the proposed solution. Finally, we designated the design methodology for our platform.

Having covered these, we can now proceed to the next chapter. This chapter will present the tools and technologies that will be required for the solution implementation as well as the project plan and the identification of the application's functionalities.

Chapter 2

Sprint 0 : Analysis and Specification of Requirements

Summary

2.1	Requirements Analysis	18
2.1.1	Identification of actors	18
2.1.2	Identification of functional requirements	19
2.1.3	Non functional requirements	20
2.2	Requirements specification	21
2.2.1	Global Use Case Diagram	21
2.2.2	Class diagram	22
2.3	Project management with SCRUM	24
2.3.1	Product backlog	24
2.3.2	Sprint and release planning	25
2.4	Global architecture of the project	26
2.4.1	Logic Architecture	26
2.4.2	Physical architecture	30
2.5	Work environment	31
2.5.1	Hardware environment	31
2.5.2	Software environment	31
	Conclusion	36

Introduction

After the general presentation of the project, we will do the analysis and specification of needs. This step is important in any software or conceptual development process because it gives us a better understanding of the system's context, and it places a solid foundation for system design. In this chapter, we will clarify the functional and non-functional requirements as well as the stakeholders. Also, we will determine the development environment, the project's use cases, and the software's global architecture.

2.1 Requirements Analysis

In this section, we will determine the involved actors and their respective roles. Then, we will present the functional and non-functional requirements.

2.1.1 Identification of actors

An actor in a use case diagram is an external entity (e.g., a person, system, or device) that communicates with the system being modeled. Actors represent roles, not people, and they interact with use cases to complete a specific goal. Actors are always outside the system. The proper identification of actors involves naming them precisely and describing their interactions with the system. [Fau+19]

In our platform, we identify three primary actors and one secondary actor. So we will focus on presenting the main actors who interact directly with the application and are required to authenticate before accessing the system. The secondary actor will also be briefly described.

The TABLE below presents the different actors in the system.

Table 2.1: Table of actors.

Actor	Description
Professor	The person accessing the platform with the role of requesting cloud resources based on quota availability.
University	Authenticated user representing the university who has the ability to manage professors, manage reservations, and consult bills.
CCK	Authenticated user representing the CCK who has the ability to manage billing, manage universities, manage reservations, and manage professors.
CCK infrastructure management System	This external actor has the role of verifying quota availability, creating the requested cloud resource based on quota availability and handle the federated login.

2.1.2 Identification of functional requirements

Functional requirements define the services and capabilities of the system ; they are the requirements that specify the system's input/output behaviors.

For our project, the functional needs are presented as follows :

A) Professor-specific features:

The application must allow the professor to:

Table 2.2: Professor Features

Feature Category	Functionality
Register and Login	- Register with a password - Log in with a password
Manage Own Requests	- Add a cloud resource reservation (via AI or forms) - Check reservation status
Manage Profile	- Activate Own Account through email - Modify Personal information
Manage Reserved Resources	- Start ECS - Stop ECS - Access ECS
Consult Usage	- Consult ECS metrics - Consult vCPU usage - Consult vRAM usage - Consult storage usage

B) University-specific features:

The university has those functionalities:

Table 2.3: University Features

Feature Category	Functionality
Login	- Log in using federated identity
Manage Professors	- Consult the professors list - Add professor - Modify professor information - Remove professor
Manage Reservations	- Approve the reservation request - Decline the reservation request
Manage Billing	- Consult bills
Consult Usage	- Consult ECS metrics - Consult vCPU,vRAM and storage usage

C) CCK-specific features:

Perform all tasks that can be carried out by the university. In addition, it will be able to:

Table 2.4: CCK Features

Feature Category	Functionality
University Management	- Consult the universities list
Manage Billing	- Consult bills - Generate bills

D) CCK infrastructure management system-specific features:

It provides these functionalities:

Table 2.5: CCK Infrastructure Management Features

Functionalities
- Verify quota availability - Create the requested cloud resource - Federated login

2.1.3 Non functional requirements

While functional requirements define the concrete features and capabilities of the new system, non-functional requirements represent quality criteria related to how those features are executed. They cover all technical, ergonomic, and aesthetic constraints that the system must meet to be developed and operate effectively. For our application, we identified the following requirements:

- **Response time** : The system should react quickly to user actions so it feels fast and smooth to use.
- **Robustness** : The system must remain stable and reliable, even under unusual load or stress, ensuring continuous availability and protecting data from loss or corruption.
- **Availability** : System accessibility is a major concern for users, as it must be available 24/7.
- **Simplicity** : The application must be easy to use, with simple, ergonomic, and user-friendly user interfaces.

- **Security** : It is crucial that the application has high-level security to restrict access to sensitive information to an authorized audience only. For this reason, all passwords in the database are encrypted.

2.2 Requirements specification

2.2.1 Global Use Case Diagram

To understand how the application works, we have included FIGURE 2.1 that shows the global use case diagram.

This use case diagram shows the key functionalities of the reservation and billing system and how users communicate with it.

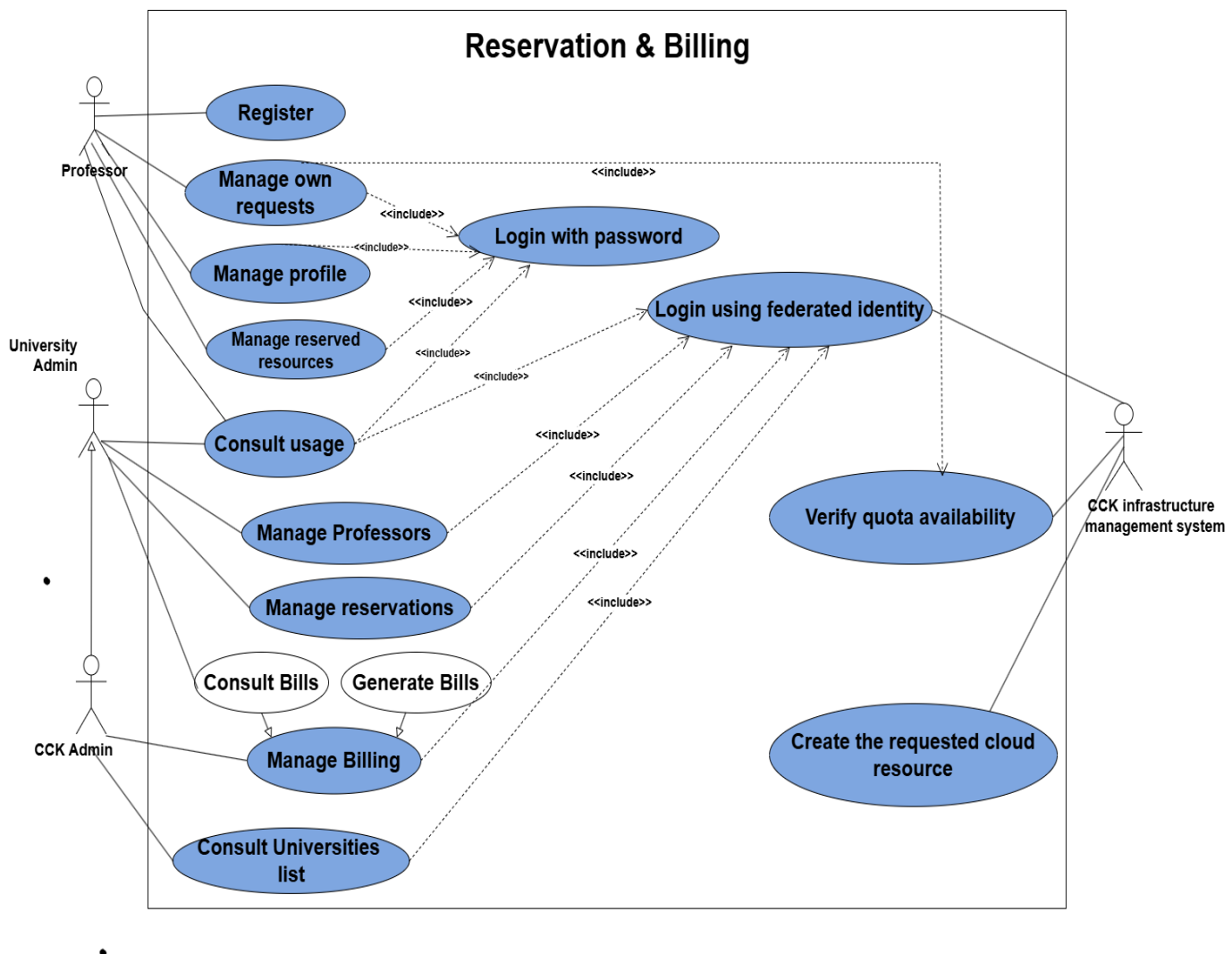


Figure 2.1: Global Use Case Diagram.

The main actors are professors, universities, CCK administrators, and the CCK infrastructure management system. Users log in to the system via Federated Login, which allows users to log in with their credentials that will be verified by the CCK infrastructure management system , or via Password Login, using user accounts that are maintained in the system of our platform.

Once authenticated:

- Professors can manage reserved resources .
- Professors can manage resource requests and profiles and can consult usage.
- University Admins manage professors and reservations and can consult bills and usage.
- CCK Admins does the same features as universities, including managing billing and quotas and consulting universities' lists.
- CCK Admins view universities data
- The CCK infrastructure management system is responsible for the federated login, verifying quota availability, and creating the requested cloud resources.

2.2.2 Class diagram

A UML class diagram represents the static view of a system; it helps to model the structure of a software system by showing the classes involved in the system, their attributes, their operations (e.g., functions/methods), and the relationships between the classes. [ASI07].

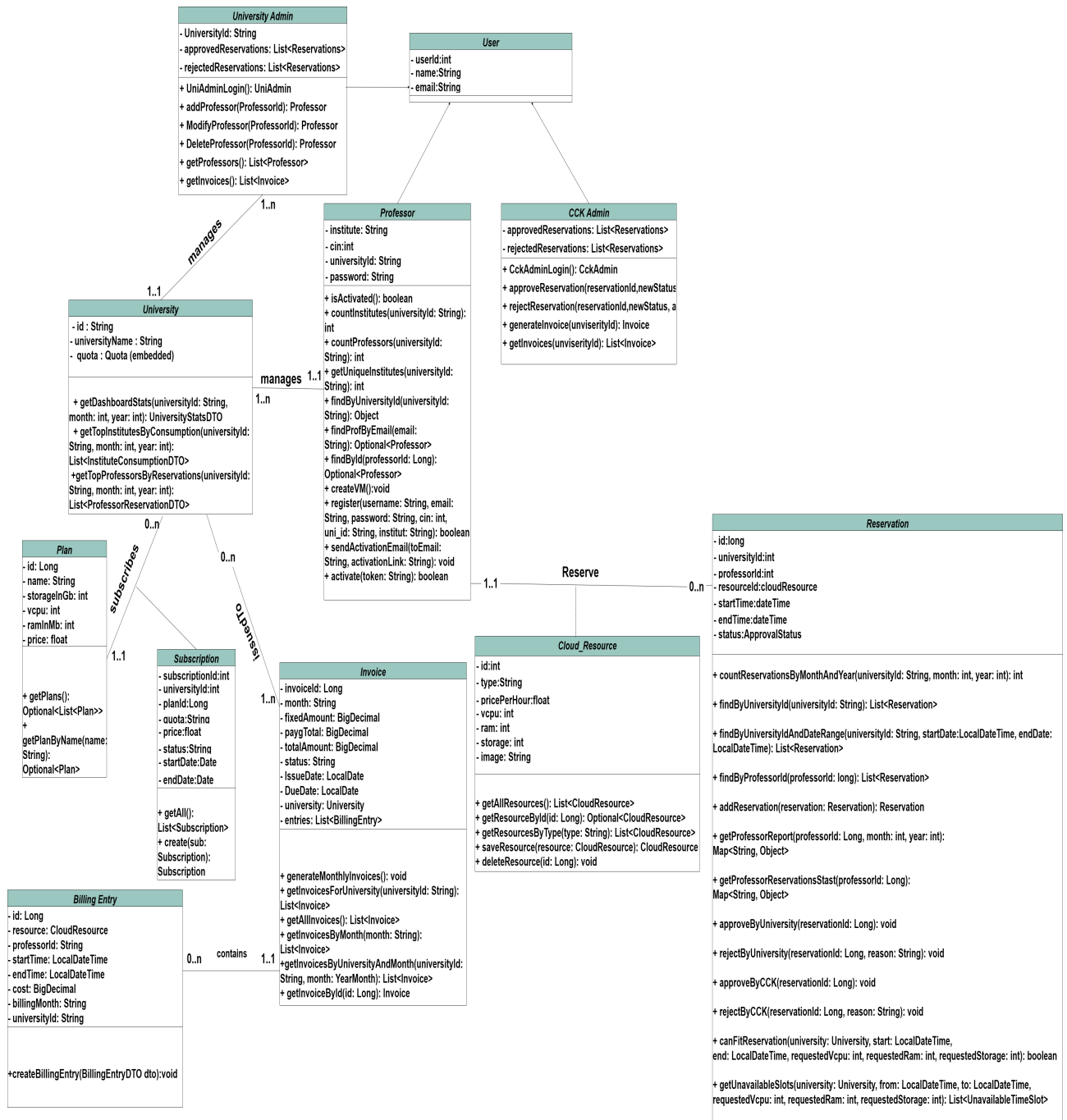


Figure 2.2: Class diagram.

2.3 Project management with SCRUM

As we mentioned in the last section, we opted for SCRUM methodology for our project. This method requires the collaboration of several stakeholders, which in our project include:

- Product Owner: Mr. Badis SAADI.
- Scrum Master : Mr. Amine KECHICHE.
- Scrum Team : Mohamed MENSI & Zaineb ARFAOUI.

2.3.1 Product backlog

The product backlog is a list of functionalities, requirements, and needs that define product vision. It is managed and prioritized by the product owner, who updates it regularly based on business value. It is written in the form of user stories to focus on user needs and maximize value. Each user story answers the questions: **who** needs it , **what** is needed, and **why** it brings value [KF].

We have started processing user stories for the highest-priority tasks. The product backlog will be used to guide the development team and can be adjusted throughout the project.

The following TABLE summarizes the application product backlog, which includes the following fields:

- **Priority:** Business value drives the prioritization of user story development based on customer expectations and needs. This priority can be high, medium, or low.
- **Complexity:** Represents the complexity of the functionality at the coding level and the various constraints to be respected, including time constraints. To assess the complexity of user stories, we opted to use a rating system on a scale of 1 to 3.
- **User Story:** It is a short description of the functionality to be done by the actor.

For the user story field, we will determine it in the following chapters.

The TABLE below presents the product backlog.

Table 2.6: Product Backlog

ID	Functionality	Priority	Complexity
1	Authentication and profile management	High	2
2	Resource Reservation	High	3
3	Usage Tracking	High	2
4	Statistics and Billing	High	3

Detailed backlog with a full feature listing can be found in the corresponding chapters.

2.3.2 Sprint and release planning

The sprint planning meeting is an important step of the scrum framework. Its primary function is to identify the sprint backlog. This planning is practically driven by the order of priorities of the product backlog, so that the most valuable and most pressing functionalities will be planned first, based on the urgency needed for the system.

The figure below illustrates the sprint planning process across four sprints, where every two sprints constitute a release, as previously defined.

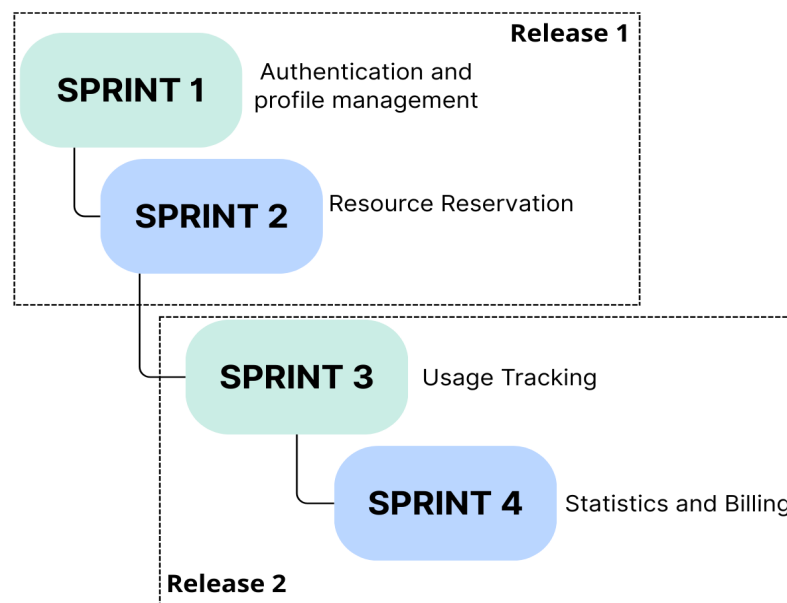


Figure 2.3: Sprints planning

2.4 Global architecture of the project

This section presents an overview of the project's overall architecture. It describes symbolically and schematically the various components of the application architecture and their interactions. We will see in this section the separation of frontend and backend responsibilities in the choice of architectural model. It also highlights the software architecture and design pattern that guide the development process.

2.4.1 Logic Architecture

✓3-tier architecture

The 3-tier architecture follows these principles:

- **Centralized data management:** all application data is stored and managed in a single, centralized database server. This helps in security and non-duplication across multiple clients.
- **Client-side presentation layer:** This layer is about displaying information and capturing user input. The user interface and user experience are handled entirely on the client side (e.g., web browser)
- **Server-side business logic:** The backend manages the core application logic. It processes user requests, applies business rules, and does the communication with the database to send or retrieve data.

This architecture offers scalability and flexibility in that it affords layers that can be changed and updated easily. [LEB03]

Its principle is to separate an application into three layers:

- **Client (Presentation Layer):** It is the interface that the user interacts with.
- **Application Server(Middleware):** It is like the bridge between the user and the data. Often called middleware, it may also communicate with other services.
- **Data server (Data Access Layer):** It is the backend system that stores and manages data.

The schematic representation 2.4 below provides a simplified illustration of the 3-tier architecture.

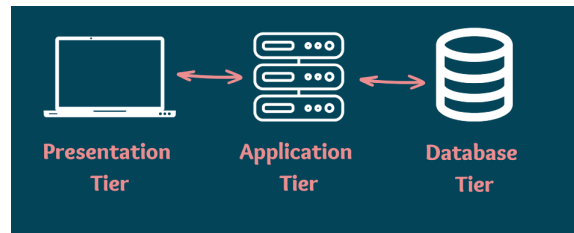


Figure 2.4: 3-tier Architecture

For our project, the components of the architecture are:

- **Frontend:** The front end is responsible for the user interface and providing a smooth user experience. The front end displays data to users and accepts their input.

We will have a front end with Angular, which follows a component-based architecture, facilitating modularity, reusability, and maintainability. The frontend communicates with the backend using RESTful APIs and accepts data interchange in JSON format.

We will also utilize Figma to prototype and visualize the user interfaces prior to a development phase during the design phase.

- **Backend:** The backend describes how the backend processes logic, handles data storage, user authentication, and API creation. It also shows how it communicates with the database and returns responses to the frontend. The structure of the backend is organized in layers (controller, service, repository) and follows MVC.

FIGURE 2.5 shows the architecture of the application.

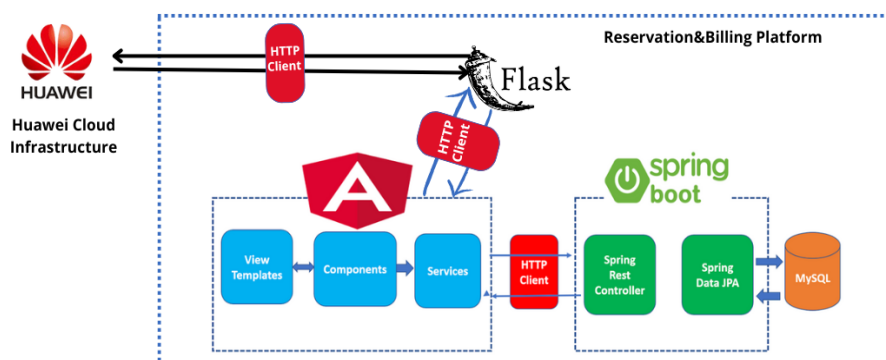


Figure 2.5: Application Architecture

✓Design pattern:

A design pattern is an approach that groups together "best practices" for solving application design problems. It is a method for structuring a computer application by breaking it down into subparts.

Our application is characterized by the MVC pattern for the backend and the MVVM pattern for the frontend.

1/ Backend Design Pattern (MVC)

The Model-View-Controller architecture is just about partitioning the application code into three separate parts, each from one of the three MVC layers:

- **Model:** The model provides a set of classes that defines the business logic of the application, allows you to define business rules with the data, and defines how the data can be processed or modified.
- **View:** The view is about rendering data to the user through the user interface. The components of the user interface are many and include, for example, HTML, CSS, and JavaScript (including frameworks like jQuery). The view is responsible for rendering the data on the user interface that it receives from the controller. The view renders the data that it receives from the model as a user interface to interact with.
- **Controller:** The controller is the intermediary that sits between the model and the view. The controller receives the data from the view, calls the appropriate functions in the model to control data/business logic, and sends the relevant data back to the view to present to the user.

The FIGURE below shows the different layers of the MVC architecture.

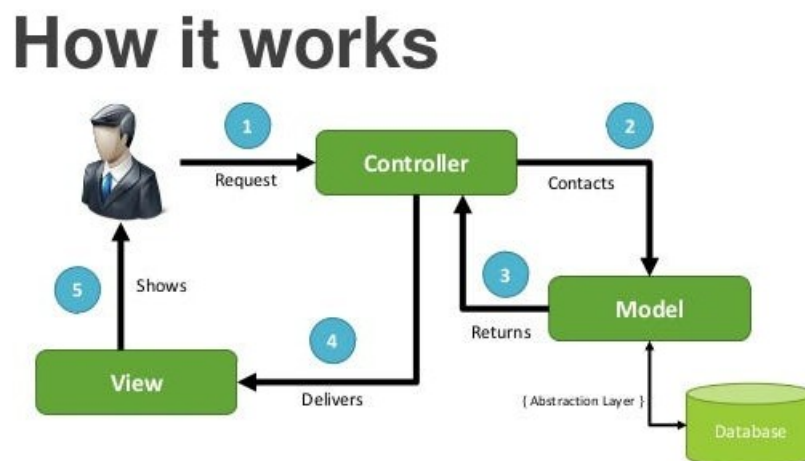


Figure 2.6: Design Pattern MVC

2/ Frontend Design Pattern (MVVM)

The structure design pattern "Model-View-ViewModel" (MVVM) enables objects to be separated into three groups:

- **Similar to the MVC architecture**, the model represents the business logic for the MVVM pattern. It is defined as a group of classes that will describe the business rules and how the data is processed or modified. The view is similar to the user interfaces (like HTML, CSS, etc.) that are responsible for delivering data to a user.
- **View:** The view is responsible for displaying the data to a user. The view is bound to the ViewModel, which will provide the data and logic that the view needs. The view will take the data and convert it into a user-friendly interface, and thanks to two-way binding (data binding), it will automatically update when the ViewModel (or Model) data changes.
- **View-Model:** The view model is similar to the bridge between the view and the model. The view model will take the data from the model and deliver it in a way that can be easily presented by the view. It holds a number of methods and commands (presentation logic) for the view to function. It also describes the state of the view and supports two-way data binding so that changes in the view or model are automatically synchronized.

The FIGURE below illustrates the different layers of the MVVM architecture.

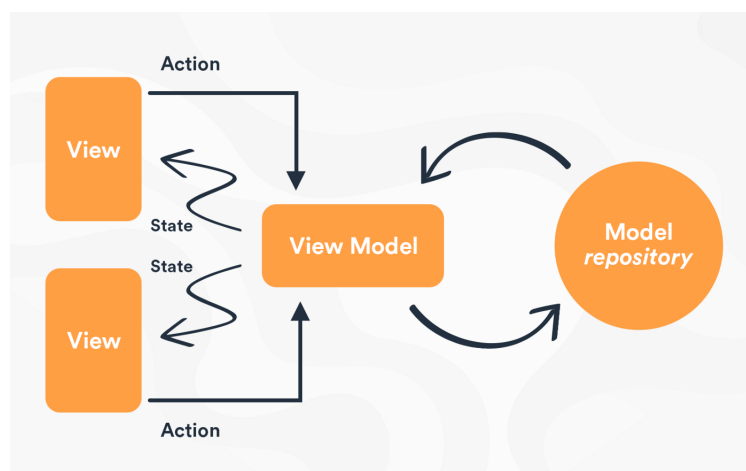


Figure 2.7: Design Pattern MVVM

2.4.2 Physical architecture

A deployment diagram is a UML diagram that presents the physical configuration of both software and hardware in a system. It is composed of nodes (e.g., servers, computers...) and communication protocols between them. This diagram makes the system clearer and more understandable. [Gor02]

The figure below illustrates the deployment diagram of our solution.

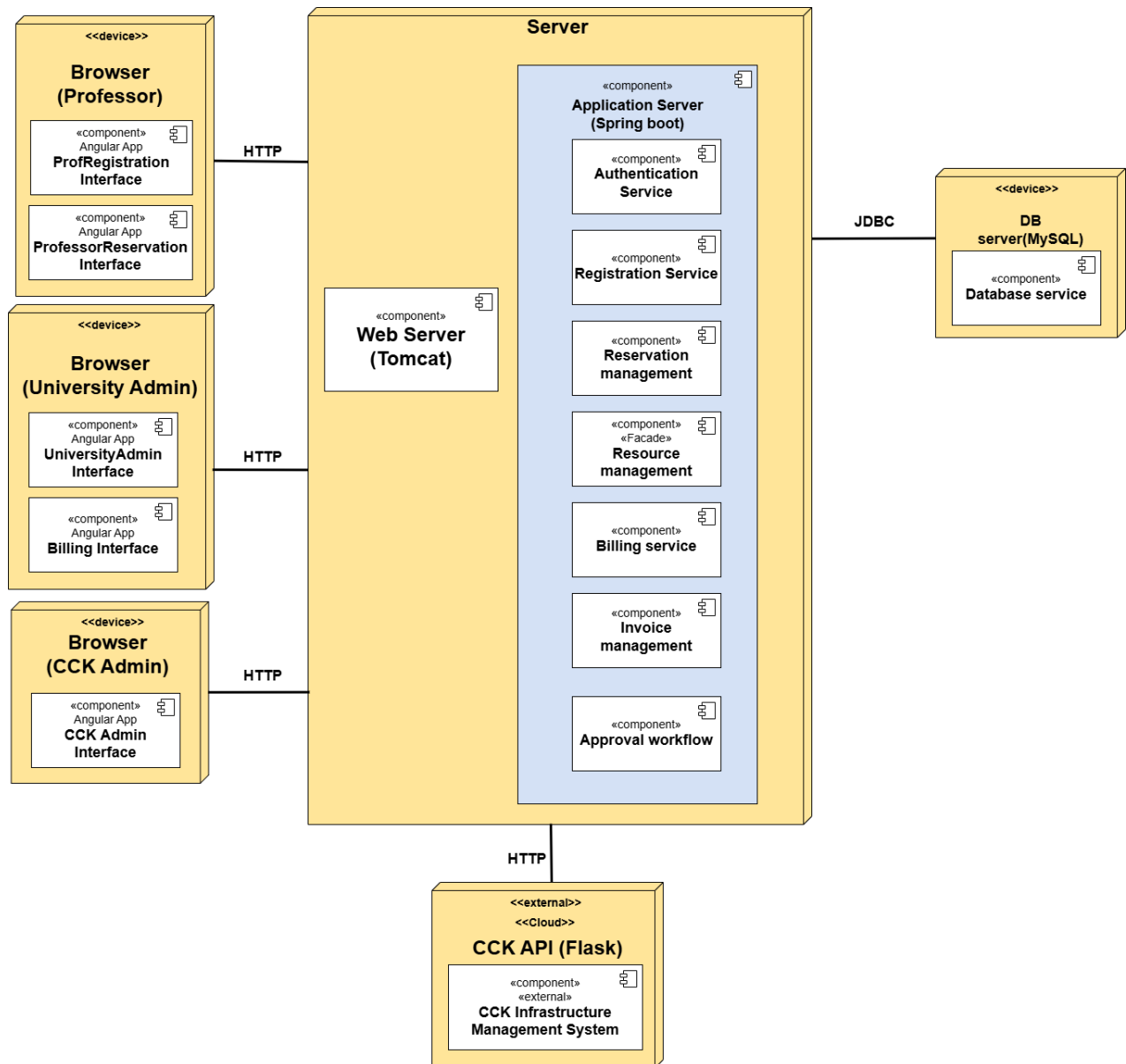


Figure 2.8: Deployment Diagram

2.5 Work environment

2.5.1 Hardware environment

To design and implement this project, a shared virtual machine (VM) was used as the primary development environment. Its configuration is detailed in the table below.

Table 2.7: Hardware Environment – Shared Development VM

Processor (vCPUs)	RAM	Storage	Operating System
4 vCPUs	16 GB	100 GB SSD	Windows Server 2019/2022

This shared development environment was used for coding, testing, and deploying the platform. It provided a centralized and consistent setup for all contributors. A relational database (MySQL) was used for managing both application and resource data, with an initial storage capacity of 500 GB and the option to scale if needed.

2.5.2 Software environment

We adopted the following tools and technologies to put our solution into reality:

Design tool

Draw.io

Draw.io (diagrams.net) is an open-source diagramming tool used for creating UML diagrams. It offers predefined templates that help developers to quickly build structured diagrams [NBG24].



Figure 2.9: Drawio Logo.

Report Writing Tool

LaTeX is a free document preparation system used to create clean, professional documents that include complex elements (e.g., mathematical equations, tables, figures). It is popular with students, researchers, and engineers who write academic papers and technical reports.

Also, Overleaf is an online LaTeX editor ; it helps users work together on LaTeX documents in real time [Sar21].



Figure 2.10: Overleaf Logo.

Testing and integration tools

Postman

Postman is an API development and testing tool that is used to send requests, see responses, and automate testing, all from one easy interface. Postman supports REST, SOAP, and GraphQL APIs. It is also great for working in a team to save time and keep API projects organized [Kor+22].



Figure 2.11: Postman Logo.

Database Management System

MySQL Workbench 8.0

MySQL Workbench is a Graphical User Interface (GUI) that allows users to interact with MySQL, which is a relational database that uses SQL commands. We could interact with MySQL databases via the terminal, but this graphical environment simplifies data visualization and database creation. [Wor13]



Figure 2.12: MySQL Logo

Technologies and Development Languages Used

Angular 18

Angular is a framework used to create fast, advanced, and dynamic single-page applications (SPAs). It is developed by Google, made from TypeScript, and it runs on the client side. Angular breaks the application into independent components, each with its own HTML, CSS, and TypeScript [CDD19].



Figure 2.13: Angular Logo.

- **Why choose Angular?**

We chose Angular 18 to develop our web application because of its popularity and numerous advantages. In the following, we will list the main advantages offered by Angular [CDD19]:

- **Component-based Architecture :** Makes applications modular, reusable, and easier to maintain.
- **Two-way data binding :** keeps the UI and the data model in sync automatically.
- **Backed by Google :** ensures security updates, long-term support, and a strong developer community.
- **High performance :** ensures scalability and performance.
- **Client-side rendering :** The browser does most of the work, taking the load off the server side and improving performance.
- **Based on TypeScript:** TypeScript provides type safety and better structure to the code, resulting in better and more reliable code that is also easier to maintain.

Flask

Flask is a web framework used to build web applications simply. Also, it is called a micro-framework as it doesn't force users to use specific libraries or tools. It is written in Python and designed to be flexible and ideal for beginner and advanced app developers who want full control over how their app works [Sin+19].



Figure 2.14: Flask Logo

TypeScript

Typescript is a JavaScript extension that simplifies the development of large-scale JavaScript applications. TypeScript provides a module system, classes, interfaces, and a gradual type system. The idea is to ease JavaScript developer transition by letting developers apply the same programming constructs while avoiding an extensive amount of TypeScript-style annotations [BAT14].



Figure 2.15: TypeScript Logo

Visual Studio Code

Visual Studio Code is a free and open-source code editor (IDE) developed by Microsoft. VS Code supports a big number of programming languages and tools thanks to the extension marketplace. It also supports autocompletion, debugging, and git commands. It is very efficient and ideal to use for both beginners and experienced developers [Che+23].



Figure 2.16: Visual Studio Code Logo

BootStrap

It is a set of practical tools intended for the design of applications and websites. This collection includes CSS and HTML codes, interactive elements such as buttons, forms and navigation tools, as well as optional JavaScript extensions [Bha15] .



Figure 2.17: BootStrap Logo

HTML

HTML (Hypertext Markup Language) is a description language (and not a programming language) that makes it possible to describe the aspect of a document, to include various information (texts, images, sounds, animations, etc.), and to establish coherent relationships between this information thanks to hypertext links [Bd00].



Figure 2.18: HTML Logo

CSS

CSS, or cascading style sheets, make it possible to define the layout of a written document in marking language like HTML [Mey06].



Figure 2.19: CSS Logo

JavaScript

JavaScript is considered a key language because it is used by web browsers, which positions it as one of the most common programming languages in the world [Cro08].



Figure 2.20: JavaScript Logo

Spring Boot

Spring Boot is a Java development framework derived from the classic Spring framework. Its main objective is to facilitate the creation of microservices, which are generally web services grouped in the form of an API.

Spring Boot makes it easier and accelerates the development of web applications and microservices using the Spring Framework [Boo20].



Figure 2.21: Spring Boot Logo

- **Why choose Spring Boot?:**

We chose Spring Boot for the development of our application due to the many advantages it offers to developers. We will cite the main ones below : [Oma]:

- **Dependency injection:** Allows you to develop modular and scalable applications by reducing dependencies between the different modules of the application.
- **Transactions management :** facilitates transaction management in applications by offering declarative transaction management.
- **Security :** Provides a security layer for applications by taking charge of the authentication and authorization of users.
- **Integration with other frameworks :** can be used with other frameworks to improve the quality and performance of applications.
- **Simplification of development :** Reduces development time by avoiding repetition of code and focusing on the business aspects of the application.
- **Abstraction of APIs:** provides abstractions for Java EE APIs, which facilitates the use of APIs and reduces the complexity of development.

Design Tool

Figma

Figma is a powerful real-time collaborative design tool accessible via web browsers or desktop apps for multiple operating systems, with most features available for free through the starter plan. [Sta22]



Figure 2.22: Figma Logo

Conclusion

In this chapter, we presented the main components of the project, including the design pattern, the development environment, and the project architecture.

Now , we will move to the next chapter, where we will introduce Release 1.

Chapter 3

Release 1

Summary

3.1	Requirements Analysis and Specification	38
3.1.1	Release 1 Backlog	38
3.1.2	Use Case Diagram of Release 1	43
3.1.3	Analysis	44
3.2	Design	46
3.2.1	Sequence Diagrams	46
3.2.2	Class Diagram of release 1	52
3.3	Realization	53
	Conclusion	56

Introduction

This chapter presents the work completed during the first release that contains sprint 1 and sprint 2, breaking down the user stories of each sprint into tasks shown in the sprint backlog. The chapter also delves into the stages of functional specification and design, followed by the implementation of interfaces during the development phase.

This release is about the "Authentication and Profile Management" and "Resource Reservation" sprints.

3.1 Requirements Analysis and Specification

The main goal of this release is to implement a functional authentication system for users before accessing the platform and a relevant resource request process for reservations.

In this section, we will present the release 1 backlog as well as its use case diagram.

3.1.1 Release 1 Backlog

After selecting the functionalities to be implemented during sprint 0, we begin the process of determining the user stories of each one and breaking them into tasks before starting work. The detailed Sprint 1 and 2 backlog is presented then in Table 3.1, containing the complexity assigned to each task, ranging from 1 (minimum) to 3 (maximum), as well as the duration for each.

Table 3.1: Release 1 Backlog

Sprint	ID	User Stories	Type	Task	Cpx	Dur.
Authentication and Profile Management	1.1	As a professor, I want to register using my university email so that I can access cloud resources.	Front-End	Design UI for registration form	2	2d
			Back-End	Build registration API (Spring Boot)		
			Back-End	Validate email format + database connection		
			Test & Design	Test and Validation + Diagrams Design		

Authentication and Profile Management	1.2	As a professor, I want to log in to the system so that I can access my dashboard.	Front-End	Design UI for login form	1	1d
			Back-End Test & Design	Build authentication API Test and Validation + Diagrams Design		
Authentication and Profile Management	1.3	As a CCK admin, I want to log in so I can access my dashboard.	Front-End	Design CCK admin interface	2	3d
			Back-End Test & Design	Implement CCK admin authentication Test and Validation + Diagrams Design		
Authentication and Profile Management	1.4	As a university admin, I want to log in to the system so that I can access appropriate features.	Front-End	Design UI for login form	3	3d
			Back-End Test & Design	Implement role-based authentication Test and Validation + Diagrams Design		
Authentication and Profile Management	1.5	As a university admin, I want to add new professors to the system so I can onboard new faculty members.	Front-End	Design professor creation form	1	1d
			Back-End Test & Design	Implement professor creation API Test professor creation functionality		

Authentication and Profile Management	1.6	As a university admin, I want to edit professor profile information so I can keep contact details and departments up-to-date.	Front-End	Design professor edit form	1	1d
			Back-End Test & Design	Implement profile update API Test profile update functionality		
Authentication and Profile Management	1.7	As a university admin, I want to reset professor passwords so I can help faculty who are locked out.	Front-End	Design password reset interface	1	1d
			Back-End Test & Design	Implement password reset functionality Test password reset flow		
Authentication and Profile Management	1.8	As a university admin, I want to deactivate professor accounts when they leave so I can maintain system security.	Front-End	Design account deactivation interface	1	1d
			Back-End Test & Design	Implement account deactivation API Test deactivation functionality		

Authentication and Profile Management	1.9	As a professor, I want to activate my account via email so I can access the system.	Front-End	Design account activation page	2	3d
			Back-End Test & Design	Implement email verification system Test email verification flow		
Authentication and Profile Management	1.10	As a professor, I want to modify my personal information (including password) so my account remains secure and up-to-date.	Front-End	Design profile edit interface	2	3d
			Back-End Test & Design	Implement profile update API Test profile update functionality		
Resource Reservation	2.1	As a professor, I want to fill a form with the resource needed (CPU cores, RAM, storage capacity, and OS selection) so that it can be reserved for a specific duration.	Front-End	Resource request form UI	3	5d
			Back-End Test & Design	Implement resource reservation API Test reservation functionality		

Resource Reser- vation	2.2	As a professor, I want to describe my needs in natural language so the chatbot can suggest the right cloud resources to request an ECS instance.	Front-End Back-End Test & Design	AI text box UI Integrate Chatbot API AI chatbot functionality	2	3d
Resource Reser- vation	2.3	As a university admin, I want to approve/reject professors' reservation requests to control resource allocation.	Front-End Back-End Test & Design	Implement approval/rejection interface Implement approval/rejection flow AI approval/rejection functionality	3	1d
Resource Reser- vation	2.4	As a CCK admin, I want to manage university reservations (PAYG) to ensure proper allocation and cost control.	Front-End Back-End Test & Design	Reservation management dashboard Dashboard + PAYG reservation control Test PAYG controls and integration	2	3d
Resource Reser- vation	2.5	As a professor, I want to access the requested ECS so I can use it for research and teaching.	Front-End Back-End Test & Design	Access button / control panel ECS access API integration Test ECS connection workflows	2	1d

Resource Reservation	2.6	As a professor, I want to start/stop ECSs to control costs and resource usage when not actively using them.	Front-End Back-End Test & Design	ECSs control UI (start/stop) ECS state management API Test ECS state transitions	2	1d
Resource Reservation	2.7	As a professor, I want to request additional resources when needed to complete my work without interruption.	Front-End Back-End Test & Design	Additional resource request form Resource expansion API Test resource scaling functionality	2	5d

3.1.2 Use Case Diagram of Release 1

In the FIGURE below you can observe the use case diagram of the first release.



Figure 3.1: Use case diagram of Release 1

3.1.3 Analysis

To better understand the approach of release 1, we provide a detailed description of the different possible scenarios.

- **Use cases:**

- Sprint 1: Authentication and profile management**

- Register and login with a password are the essential authentication use cases to allow user access.

- Register includes account creation via password, requiring validation of university email.
- Login with password provides secure access for professors and differentiates that process from login with federated identity, which is for university and CCK admins.
- When professors log in, they can access Manage Profile (including activating their account and changing their personal information).

Sprint 2: Resource Reservation

- Manage reserved resources, which allows professors to (Access requested ECS, Start or stop the ECS).
- The system supports two reservation flows: add a Reservation using AI, using a natural language interface to simplify cloud requests and add Reservation using manual forms, which allows for traditional input by using structured fields along with checking reservation status.
- Both flows are specializations of Add a cloud resource reservation, and are dependent on prior authentication, and the verification of quota availability.
- Manage reservations includes administrative actions such as approve or decline reservation requests.
- Consult the universities list: CCK admins view university data.

- **Preconditions:**

- **For Registration:** the user must be a professor with a valid university email address, username, and password.
- **For Login:** Users must have appropriate credentials based on their role (professor, university admin, or CCK admin).
- **For federated Login:** The CCK infrastructure management system must be operational and provide federated login services.
- **For Management Functions:** Users must be successfully authenticated and authorized according to their specific role.
- User must be authenticated as a professor (via the login use case).
- Professor must have an available quota for requested resources (verified via "verify quota availability").
- Reservation System must be accessible to receive and process requests.

- **Main Scenarios:**

- **Professor Registration:** Professor provides a university email, username, and password, and an account is created after verification.

- **Login Authentication with password:**
If the user is a professor, he must provide the right credentials.
- **Federated Authentication:**
Admin user is redirected to CCK infrastructure for validation and redirected back upon success.
- **Professor makes a reservation using AI.**
- **Professor makes a reservation using manual form.**
- **Professor checks their reservation status.**
- **Professors manage their profiles.**
- **Professor manages their own reserved requests (start,stop and access ECS).**
- **Postconditions:**
 - **Successful Registration:** Professor account is created.
 - **Successful Login:** User is redirected to his dashboard.
 - **Successful Reservation:** Cloud resources are allocated and reserved for the professor.
 - **Resource Record:** System maintains a record of resource allocation connected to the professor's account.

3.2 Design

The design phase includes a key step that supports implementation and testing activities, which in our case is represented by the class diagram and sequence diagrams for Sprint 1.

3.2.1 Sequence Diagrams

A sequence diagram is a type of Unified Modeling Language (UML) diagram that illustrates the sequence of messages exchanged between objects during an interaction. It consists of a set of objects, represented by lifelines, and the messages exchanged between these objects during that interaction [ASK05].

The image below represents the Professor Registration Sequence Diagram.

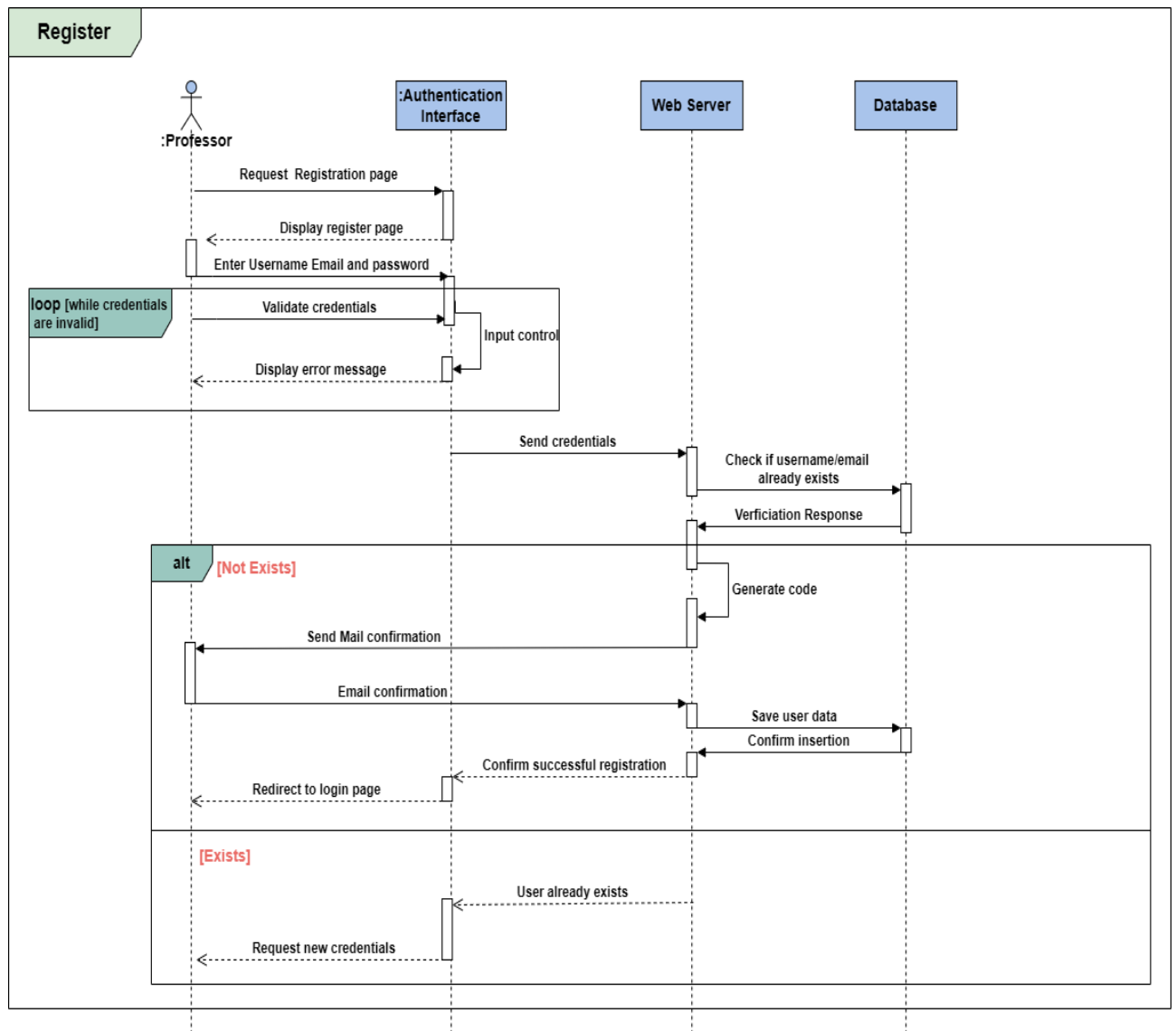


Figure 3.2: Professor Registration Sequence Diagram.

The diagram shows a sequence of actions for professors to register. Initially, it begins with a request for the registration page from the Authentication System. The system then presents the registration interface for professors to input their credentials.

To ensure quality, the web server shows a validation loop that checks on the user's information and can show error messages if the validation fails. On a successful validation, the web server will initiate a database check to see if the username/email already exists. There are two possible paths after that:

- If they do not exist, the web server generates a verification code, a confirmation email is sent to the user's email address, and the user data is saved to the database after email confirmation. The system then redirects the user to the login page.
- If they exist, the web server notifies the authentication system, and the user re-enters credentials.

The two sequence diagrams illustrate authentication flows for the user types of university, CCK administrators, and professors.

- The user initiates the authentication request to the authentication system.
- All user types need to enter their credentials, followed by an input control that will validate the credentials before server communication occurs.
- The credentials will be sent to the database/IAM API for identity verification.
- Successful authentication in all flows results in token generation and storage.
- In all these diagrams, there were alternative (alt) fragments to handle the two different responses from the authentication:

- Response Success: provided valid credentials, created authentication tokens and matched with dashboard access.
- Response Failed: Invalid credentials are provided, and error messages are shown.

- Dashboard Redirect: when authenticated, all users are redirected to their own dashboards on completion of logging in.

The image below represents the Professor Login Sequence Diagram.

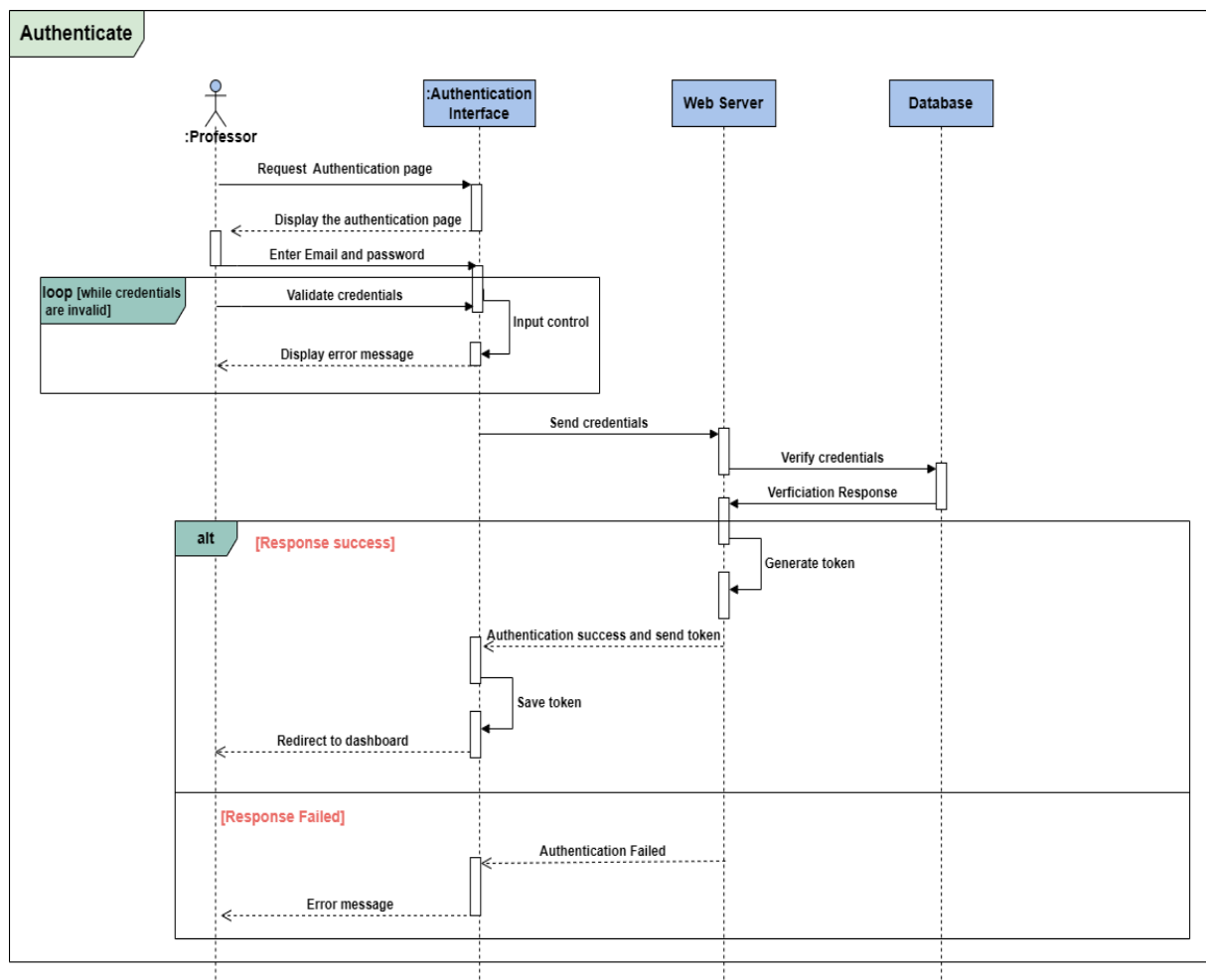


Figure 3.3: Professor Authentication Sequence Diagram

The image below represents the CCK Admin and university admin Login Sequence Diagram.

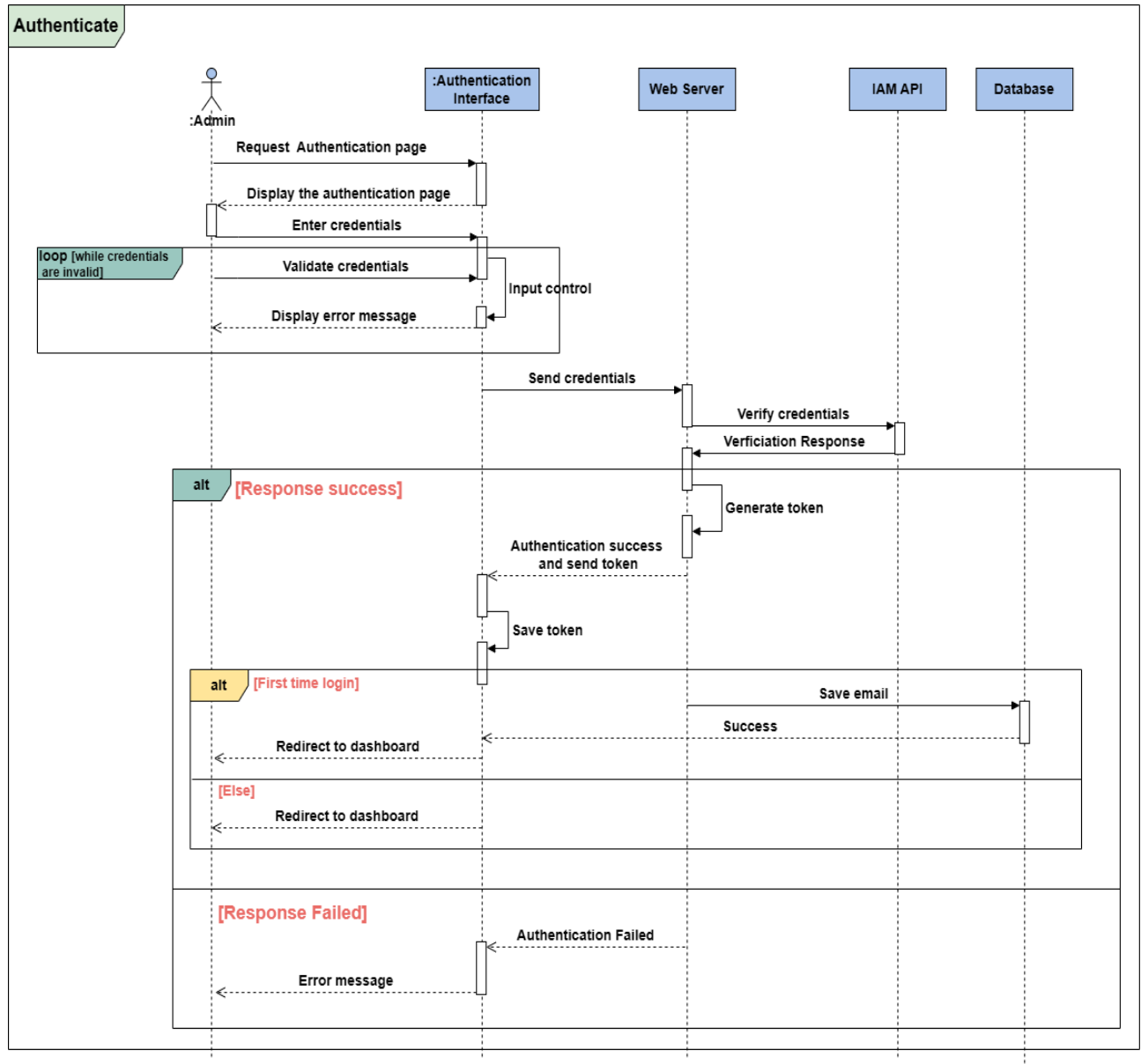


Figure 3.4: Admin Authentication Sequence Diagram.

The sequence diagrams provide additional clarity on top of functionality in the use case diagram by detailing critical implementation facts. The diagrams show that abstract use cases are completed by concrete interactions within the system. The diagrams also illustrate how workflow of "Add a cloud resource reservation" actually diverges into two separately implemented processes (an AI-assisted process and a form-based process) and that it involves multiple approval stages depending on resource availability.

The image below represents the Reservation approach Sequence Diagram.

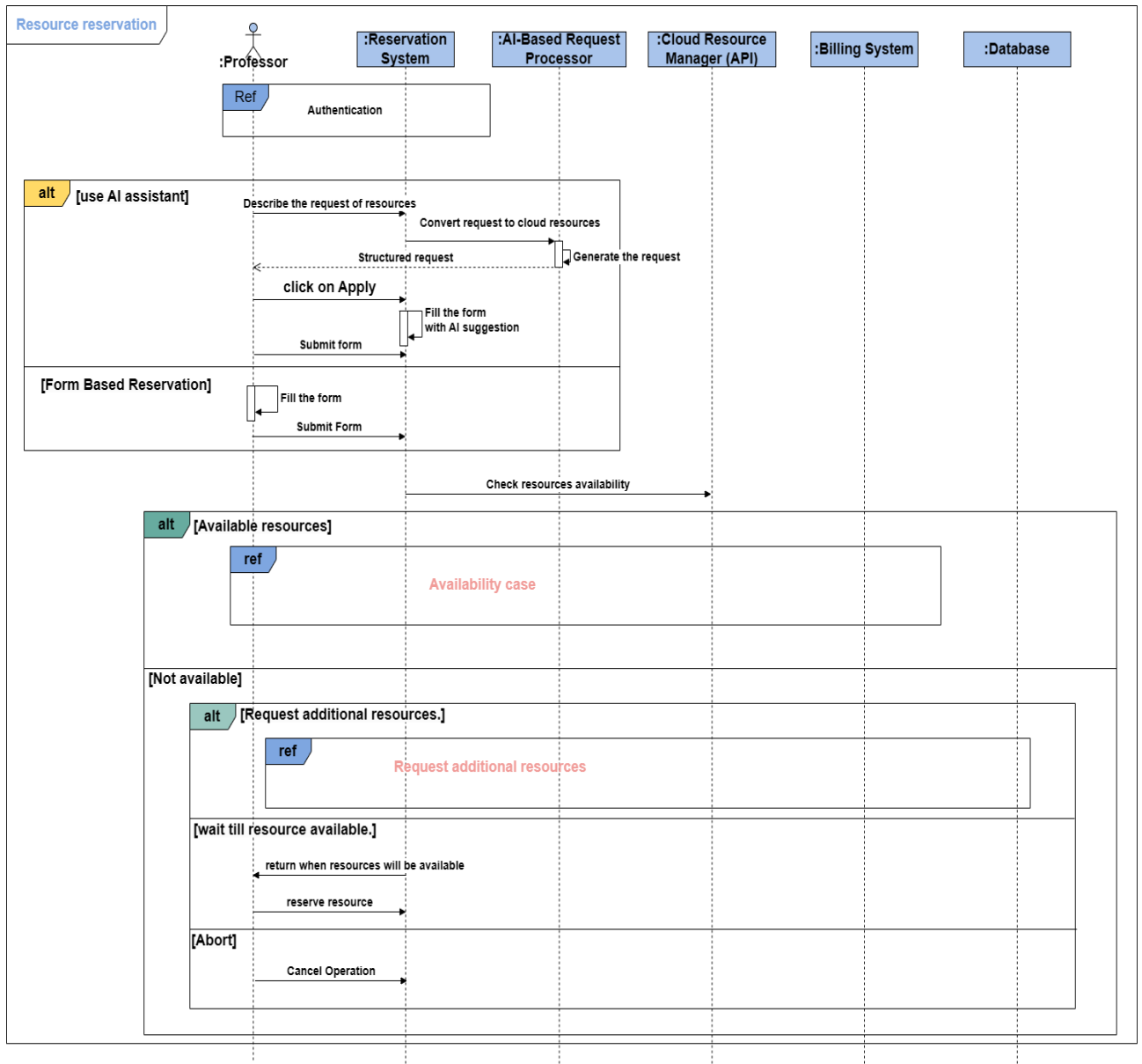


Figure 3.5: Reservation approach Sequence Diagram

The image below represents the Request Additional Resources Sequence Diagram.

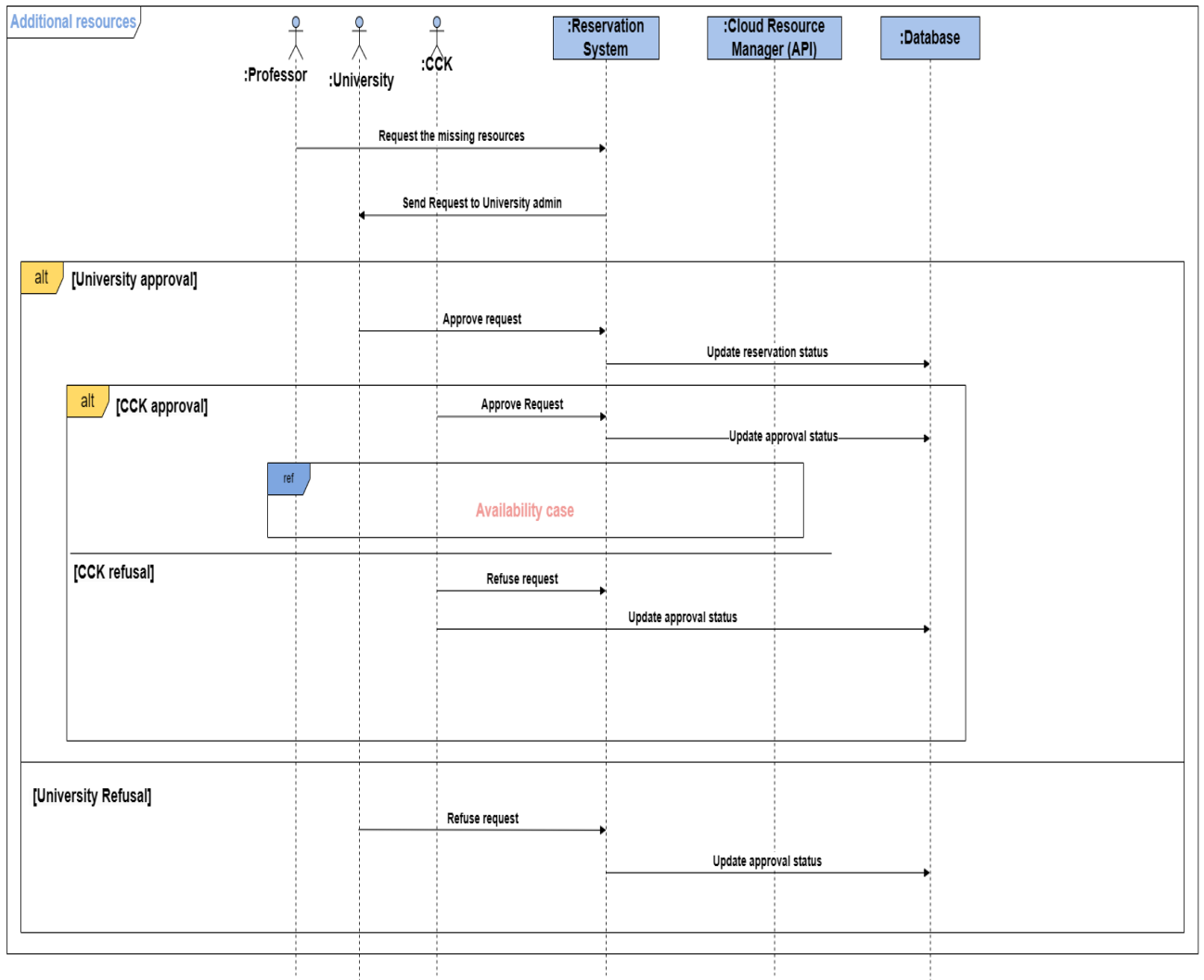


Figure 3.6: Additional Resources Sequence Diagram.

The image below represents the Available Resources Sequence Diagram.

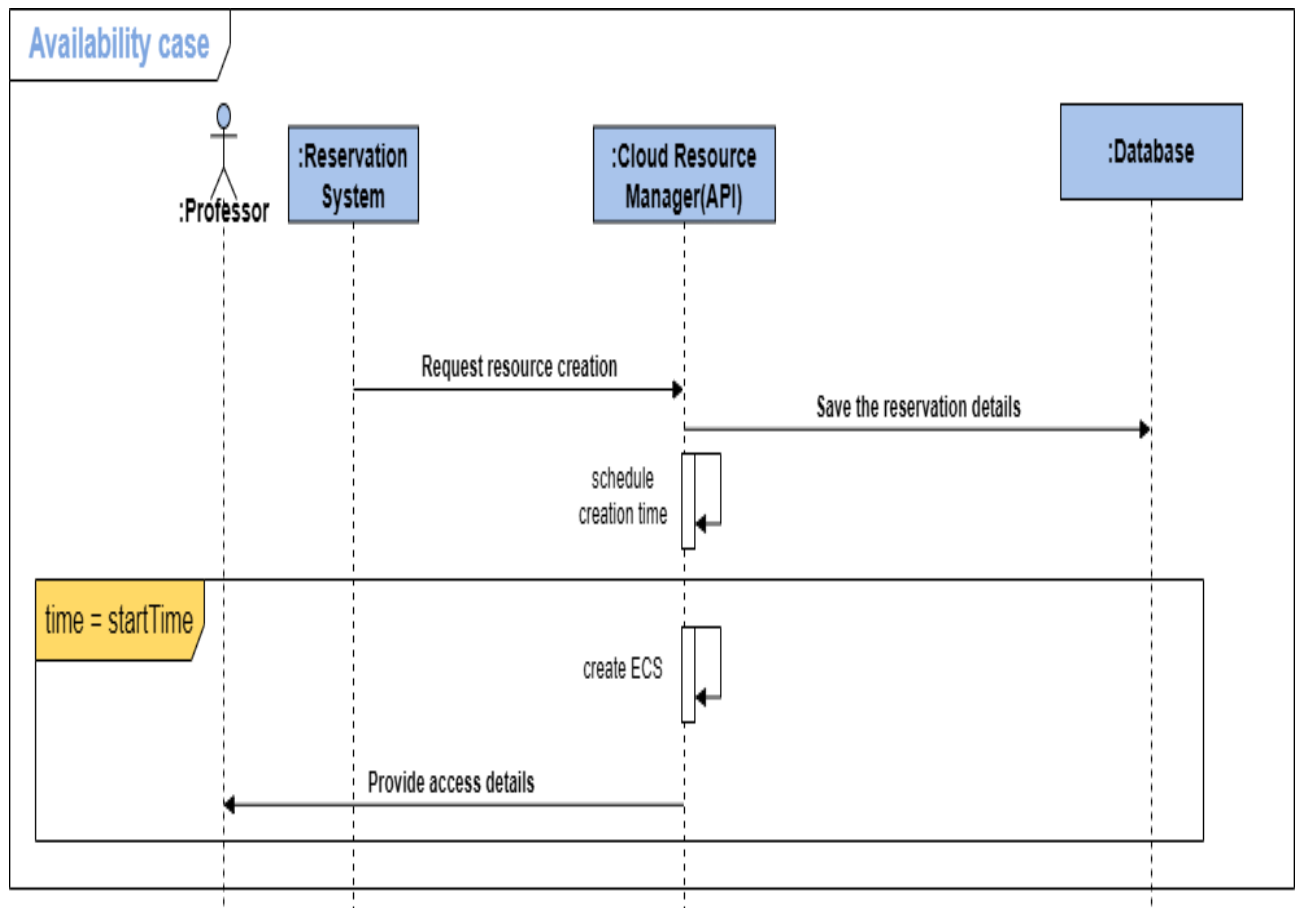


Figure 3.7: Available Resources Sequence Diagram.

3.2.2 Class Diagram of release 1

A class diagram represents the static structure of a software system, showing classes, their attributes, their methods, and the relationships between them.

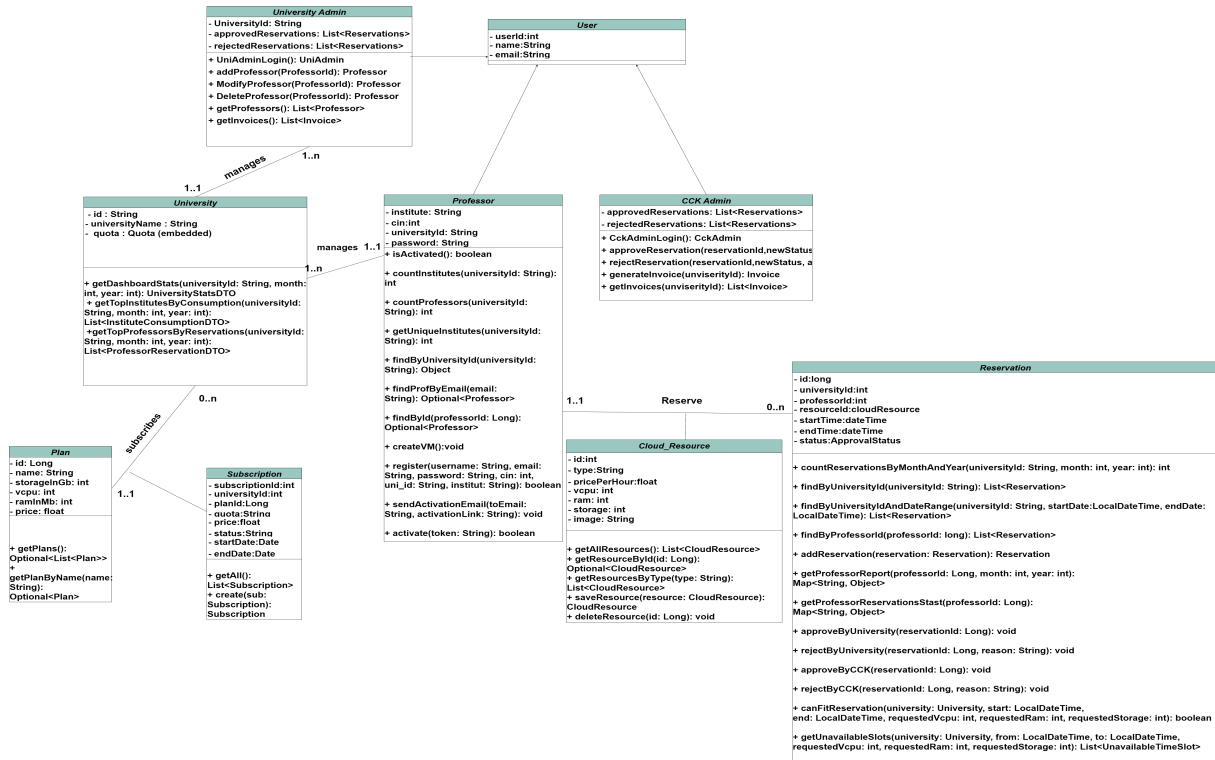


Figure 3.8: Class Diagram of release 1

3.3 Realization

In the following, we will present some interfaces related to release 1 of our application.

Register

Username

Email

Password

Register

Already have an account? [Login here](#)

Login

[Professor Login](#)

[University Login](#)

Email

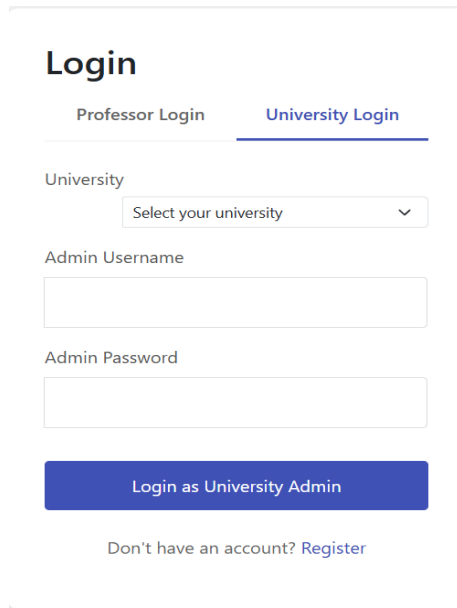
Password

Login as Professor

Don't have an account? [Register](#)

Figure 3.9: Interface: Professor Registration
The form where professors can register by entering a username, email, and password.

Figure 3.10: Interface: Professor Login
Allows professors to log in to their account using registered credentials.



Login

Professor Login **University Login**

University

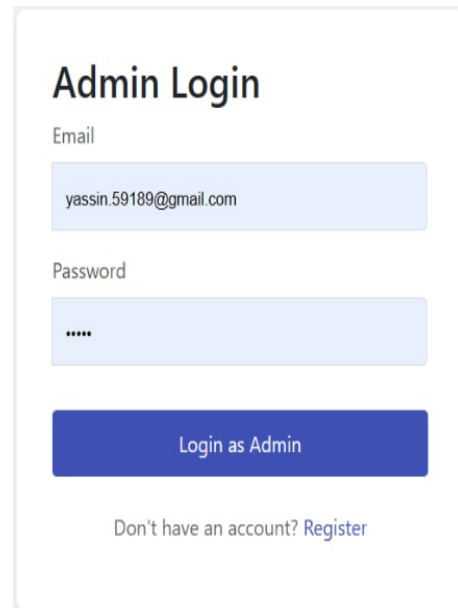
Admin Username

Admin Password

Login as University Admin

Don't have an account? [Register](#)

Figure 3.11: Interface: University Login
 Used by university admins to log in by selecting their university and entering credentials.



Admin Login

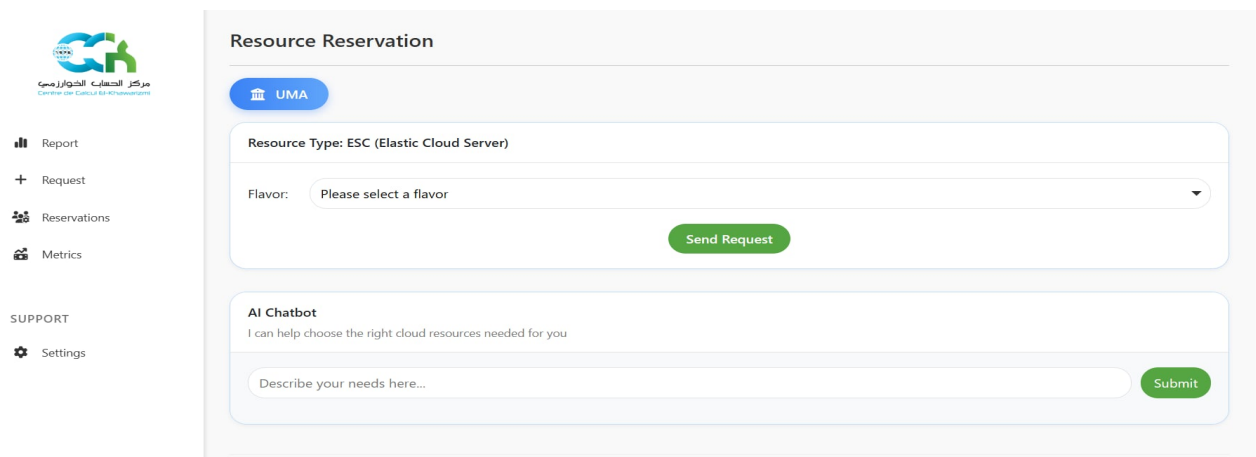
Email

Password

Login as Admin

Don't have an account? [Register](#)

Figure 3.12: Interface: CCK Login
 CCK administrators use this page to access the management dashboard securely.



Resource Reservation

UMA

Resource Type: ESC (Elastic Cloud Server)

Flavor:

Send Request

AI Chatbot
 I can help choose the right cloud resources needed for you

Submit

Figure 3.13: Interface: Resource Reservation
 Professors can request available resources by choosing type, duration with the help of an AI assistant.



Update Profile

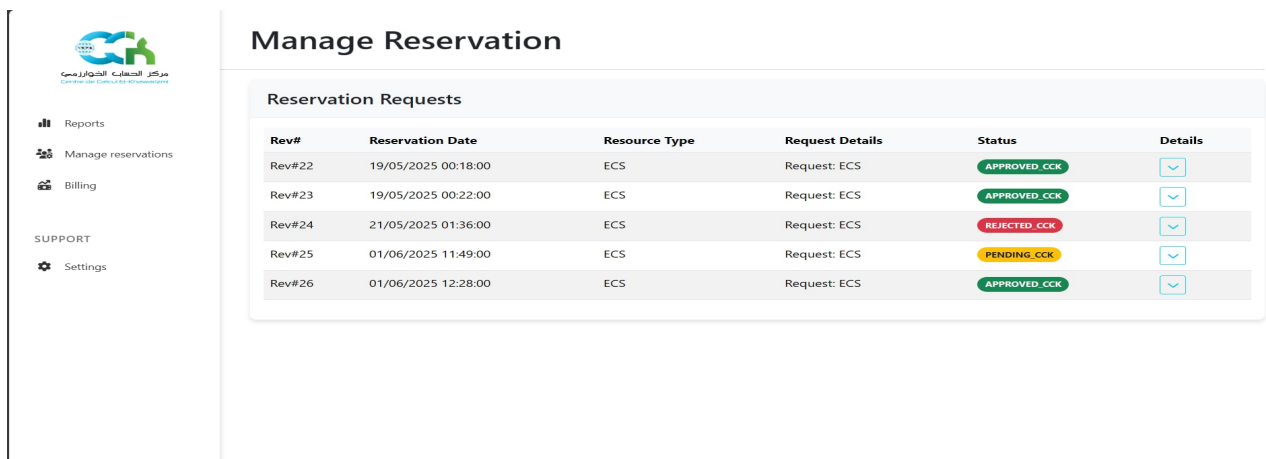
Email:

Name:

Institute:

University:

Figure 3.14: Interface: Manage Profile for Professor
Professors can update their personal information and preferences.

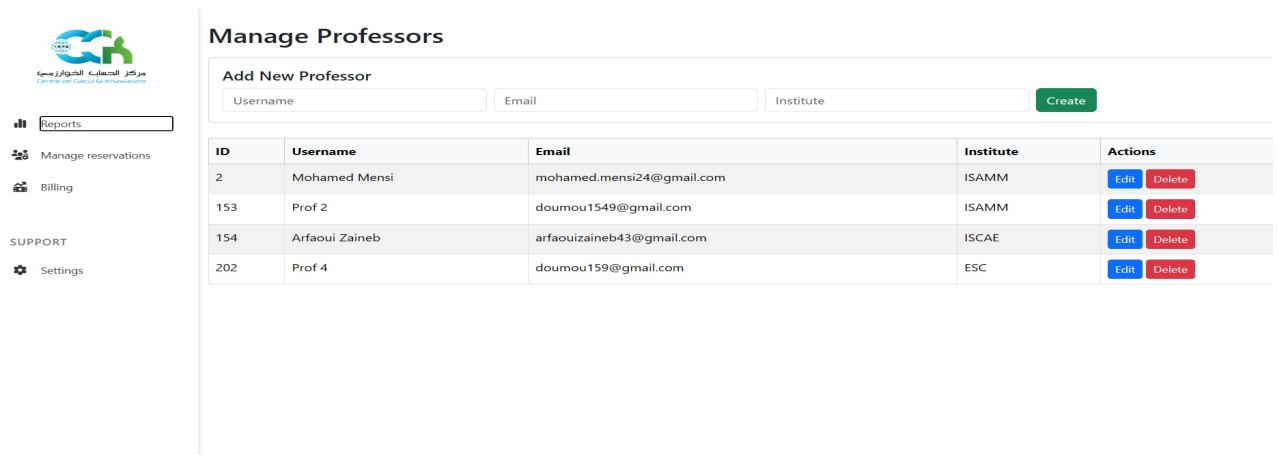


Manage Reservation

Reservation Requests

Rev#	Reservation Date	Resource Type	Request Details	Status	Details
Rev#22	19/05/2025 00:18:00	ECS	Request: ECS	APPROVED_CCK	v
Rev#23	19/05/2025 00:22:00	ECS	Request: ECS	APPROVED_CCK	v
Rev#24	21/05/2025 01:36:00	ECS	Request: ECS	REJECTED_CCK	v
Rev#25	01/06/2025 11:49:00	ECS	Request: ECS	PENDING_CCK	v
Rev#26	01/06/2025 12:28:00	ECS	Request: ECS	APPROVED_CCK	v

Figure 3.15: Interface: University Manage Reservations
Universities can view, approve, or reject reservation requests submitted by professors.



Manage Professors

Add New Professor

Username Email Institute

ID	Username	Email	Institute	Actions
2	Mohamed Mensi	mohamed.mensi24@gmail.com	ISAMM	Edit Delete
153	Prof 2	doumou1549@gmail.com	ISAMM	Edit Delete
154	Arfaoui Zaineb	arfaouizaineb43@gmail.com	ISCAE	Edit Delete
202	Prof 4	doumou159@gmail.com	ESC	Edit Delete

Figure 3.16: Interface: University Manage Professors
University admins manage professor accounts(add,modify and delete).



ID	University	Professor	Resource	Duration	Status	Details
#25		Prof 2	ECS	24 hrs 0 mins	PENDING CCK	

Figure 3.17: Interface: CCK Manage Reservations

CCK can view all university reservation activity and approve or reject as needed.

Conclusion

In this chapter, we outlined the Release 1 backlog, as well as the design, analysis, and interfaces for the use cases. In the next chapter, we will cover the second release.

Chapter 4

Release 2

Summary

4.1	Requirements Analysis and Specification	58
4.1.1	Release 2 Backlog	58
4.1.2	Use Case Diagram of Release 2	61
4.1.3	Analysis	62
4.2	Design	63
4.2.1	Sequence Diagrams	63
4.2.2	Class Diagram of release 2	65
4.3	Realization	65
	Conclusion	68

Introduction

In this chapter, we will start our second release with the creation of a sprint backlog, then the development of a detailed use case diagram. Finally, we will analyze and design. This release is about "Usage Tracking" and "Statistics and Billing".

4.1 Requirements Analysis and Specification

In this section, we will present the sprint backlog for sprints 3 and 4 as well as the use case diagram.

4.1.1 Release 2 Backlog

Table 4.1: Release 2 Backlog

Sprint	ID	User Stories	Type	Task	Cpx	Dur.
Usage Tracking	3.1	As a Professor, I want to consult my reservations and ECS metrics through visual dashboards with charts, so I can easily track and understand my resource usage.	Front-End	Design dashboard UI with ECS and reservation metrics	2	2d
			Back-End	Create APIs for ECS usage and reservations retrieval		
			Test & Design	Test and Validation + Diagrams Design		

Usage Tracking	3.2	As a University admin, I want to monitor professors' ECS usage and quantity via charts and dashboards, so I can track resource consumption and respond quickly to peaks.	Front-End	Build dashboard for ECS usage per professor	2	2d
			Back-End	Implement ECS aggregation logic and endpoints		
			Test & Design	Test and Validation + Diagrams Design		
Usage Tracking	3.3	As a University admin, I want to receive usage alerts when total quota is being consumed and see usage trends in chart form, so I can act before resources run out.	Front-End	Display trend charts and alert indicators	2	2d
			Back-End	Implement quota threshold alerting logic		
			Test & Design	Test and Validation + Diagrams Design		
Usage Tracking	3.4	As a CCK admin, I want to receive alerts when universities exceed quotas and visualize trends through usage charts, so I can manage capacity and billing proactively.	Front-End	Design alert dashboard and trend visualization	2	2d
			Back-End	Build quota monitoring service with alert triggers		
			Test & Design	Test and Validation		

Usage Tracking	3.5	As a CCK admin, I want to view usage data of universities through charts and dashboards so I can monitor system utilization and plan capacity accordingly.	Front-End	Build cross-university usage comparison dashboards	2	3d
			Back-End	Develop endpoints for university-level usage data		
			Test & Design	Test and Validation + Diagrams Design		
Statistics and Billing	4.1	As a user, I want to access role-specific dashboards with statistics so I can view relevant metrics.	Front-End	Build dashboard UI for different roles	3	7d
			Back-End Test & Design	Implement role-based metrics APIs Test and Validation + Diagrams Design		
Statistics and Billing	4.2	As a CCK admin, I want bills to be generated automatically so universities can be charged appropriately.	Front-End	Display generated billing data in UI	2	4d
			Back-End Test & Design	Implement billing logic and schedule generation Test and Validation + Diagrams Design		
Statistics and Billing	4.3	As a university admin, I want to consult and export invoice details.	Front-End	Create UI to consult and export invoice data	2	5d
			Back-End Test & Design	Provide invoice data API with export functionality Test and Validation + Diagrams Design		

Statistics and Billing	4.4	As a CCK admin, I want to export invoices in Excel format for further analysis and accounting integration.	Front-End Back-End Test & Design	Add Excel export button in invoice UI Generate invoice export in Excel format Test and Validation + Diagrams Design	2	1.5d
Statistics and Billing	4.5	As a university admin, I want to receive invoices for additional usage.	Front-End Back-End Test & Design	Design view for additional usage invoice display Generate invoice logic based on extra usage Test and Validation + Diagrams Design	2	4d
Statistics and Billing	4.6	As a CCK admin, I want to consult a billing dashboard to track revenue and identify usage patterns across universities.	Front-End Back-End Test & Design	Build billing dashboard with usage visualizations Provide revenue and usage pattern endpoints Test and Validation + Diagrams Design	2	4d

4.1.2 Use Case Diagram of Release 2

In the FIGURE below you can observe the use case diagram of the second release.

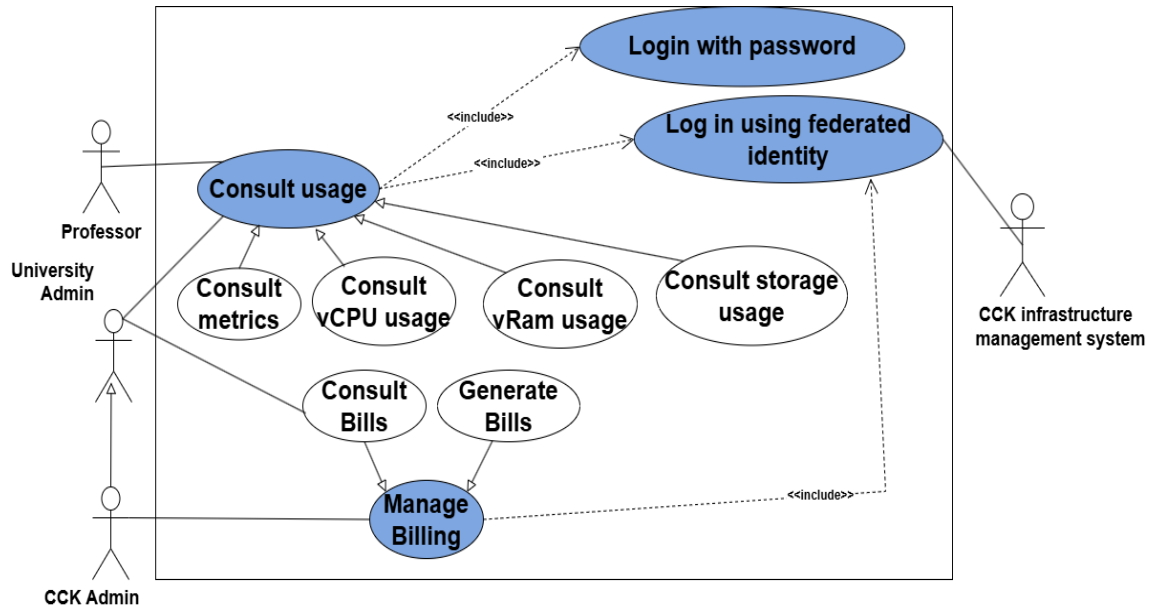


Figure 4.1: Use case diagram of Release 2

4.1.3 Analysis

To better understand the approach of release 2, we provide a detailed analysis.

- **Use cases:**

- **Consult usage:** Professors view usage dashboards and metrics.
- **Manage billing:** CCK admins generate and consult bills.

- **Preconditions:**

- Users must be authenticated via an appropriate login method.
- University has an active subscription in the system.
- Professor account is active in the system.
- Professors must have existing reservations to track.

- **Main Scenarios:**

- Users log in via password or federated identity based on role.
- Professors view dashboards showing their resource consumption.
- University admins monitor quota usage.
- University and CCK admins consult automatically generated bills.

- **Postconditions:**

- All billing information is retrievable.

- Invoices are generated and delivered successfully.
- Information visibility for the appropriate user.

4.2 Design

4.2.1 Sequence Diagrams

The image below represents the Generate Bills sequence diagram.

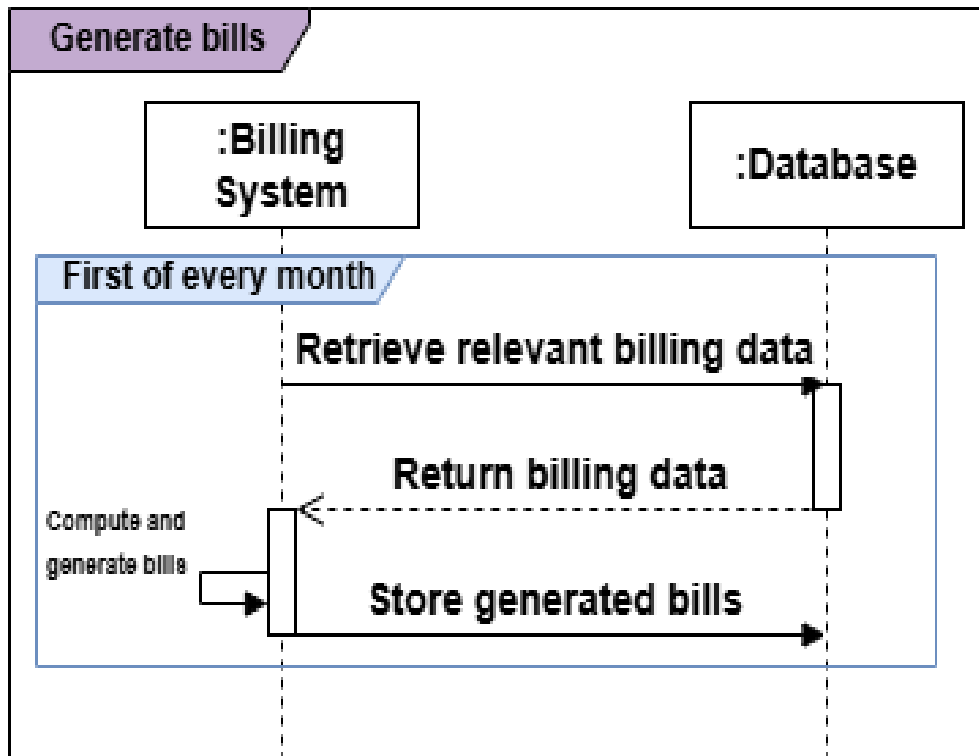


Figure 4.2: Generate bills Sequence Diagram

This sequence diagram illustrates the functionality of the "Generate Bills" found in the use case diagram under CCK admin responsibilities. While the use case diagram indicates that only CCK admins may generate bills, the sequence diagram illustrates that the action is automated and initiated by the billing system every month rather than an action directed by the CCK admins. It also reveals the logic and complexity involved in bill generation, including data retrieval and storage.

The image below represents the Consult Bills sequence diagrams.

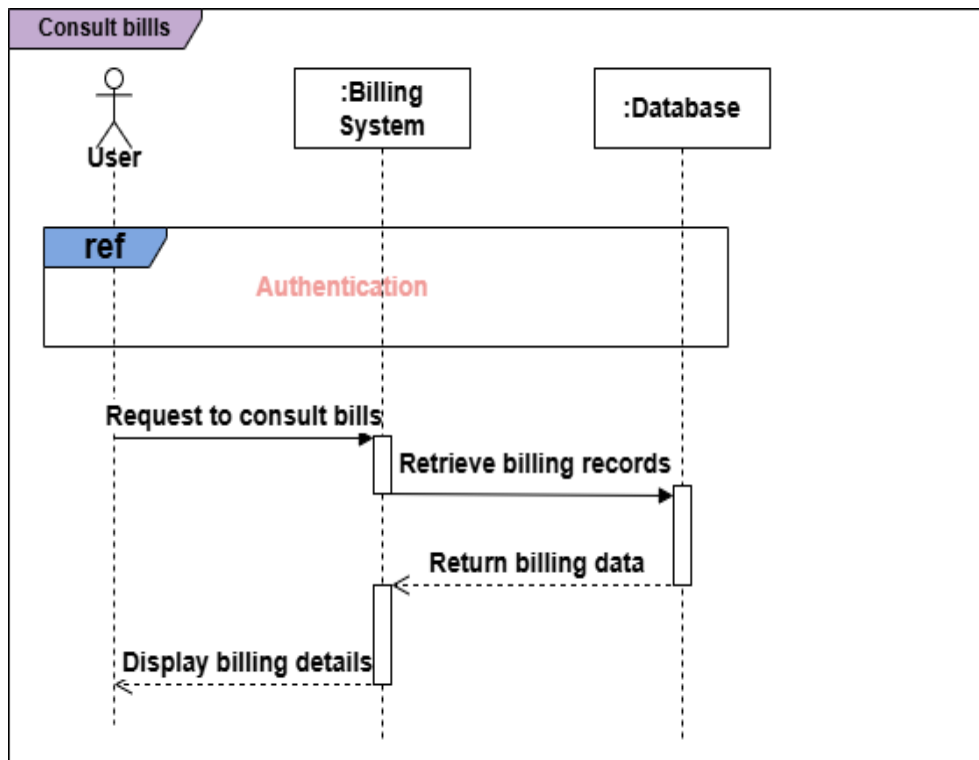


Figure 4.3: Consult bills Sequence Diagram

The sequence diagram goes into detail about one of the use cases, "Consult Bills," and it gives some implementation details that are not contained in the use case diagram. It illustrates the consultation of bills as a three-phased process: request for bill information, retrieving matching records from the database, and formatting the records to display to the user.

4.2.2 Class Diagram of release 2

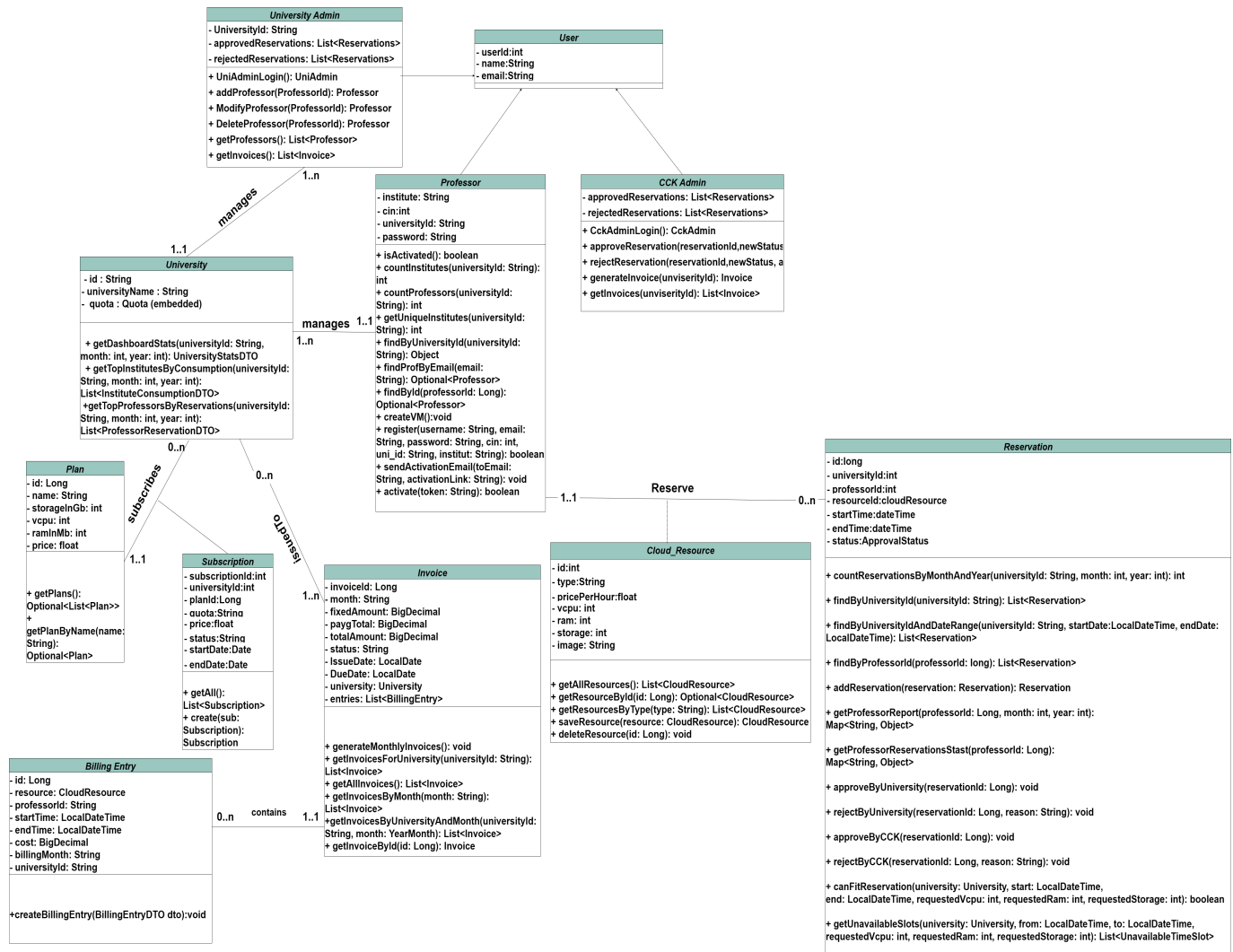


Figure 4.4: Class Diagram of Release 2

4.3 Realization

In the following, we will present the interfaces related to release 2 of our application.

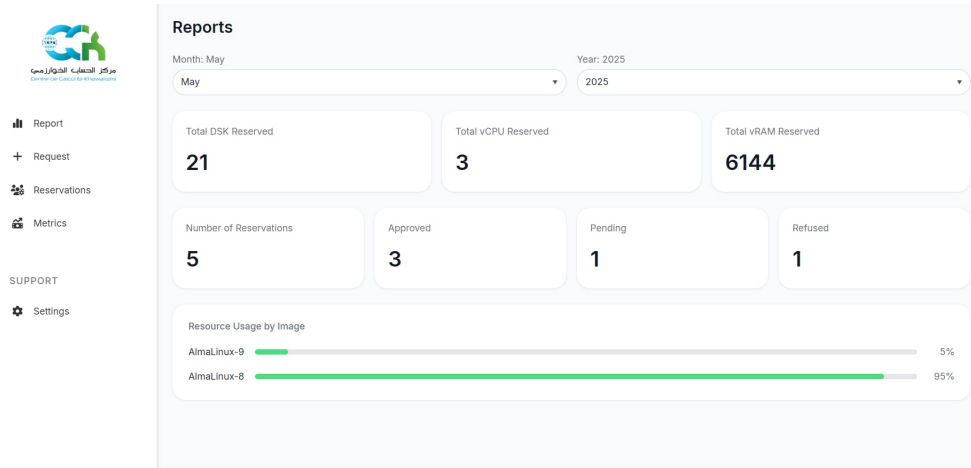


Figure 4.5: Interface: Professor Reports

Professors can consult their reserved cloud resources filtered by month and year.

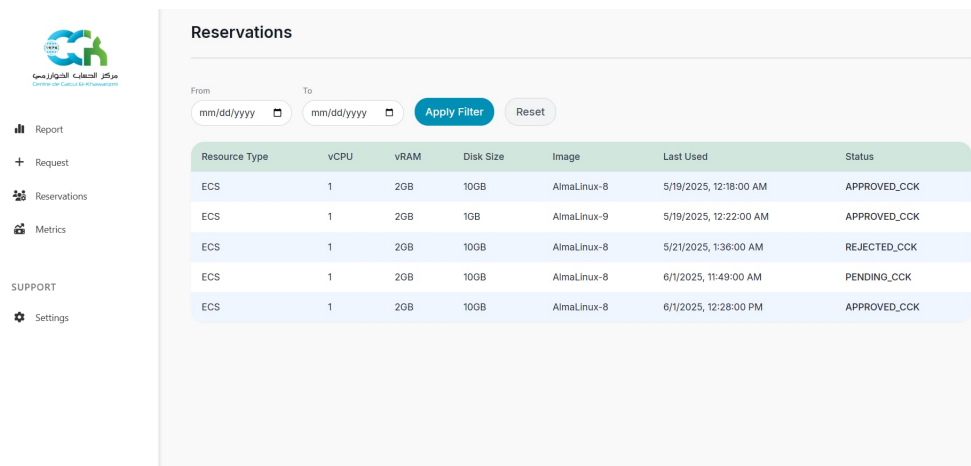


Figure 4.6: Interface: Professor Reservations

Professors can consult their reservation status if it is approved, rejected, or pending by CCK.

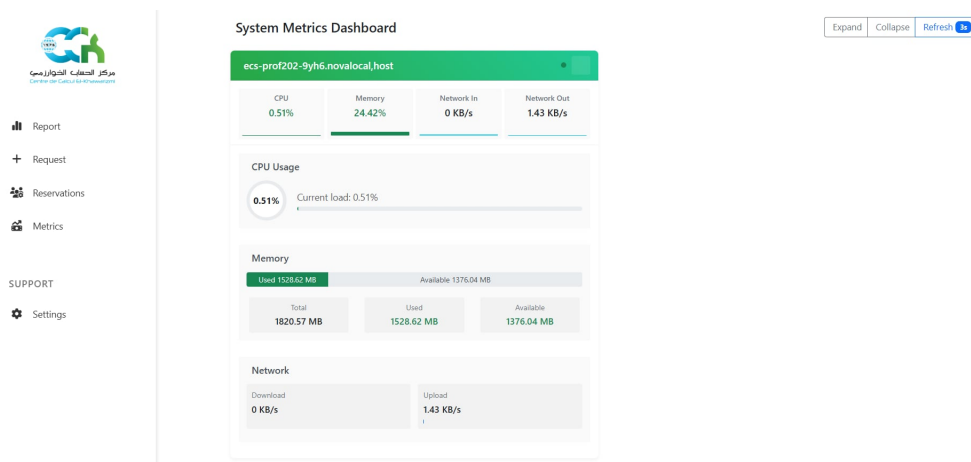


Figure 4.7: Interface: Professor Metrics

Metrics are consulted by professors to track their usage percentage.

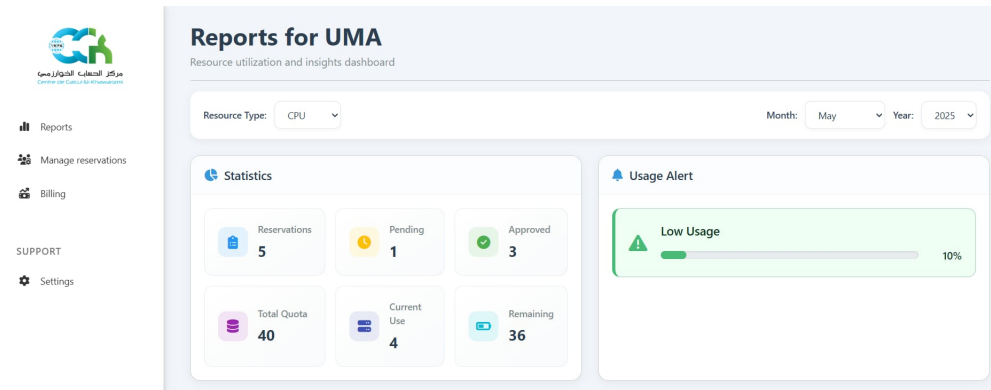


Figure 4.8: Interface: University Reports

Universities can consult their resource statistics with a month/year and resource type filters.

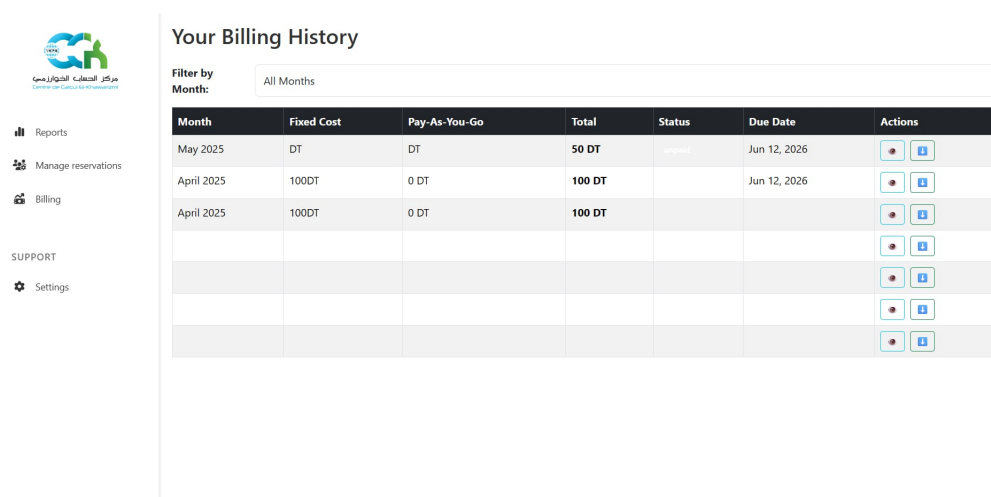


Figure 4.9: Interface: University Billing

Universities can track their billing history in every month.

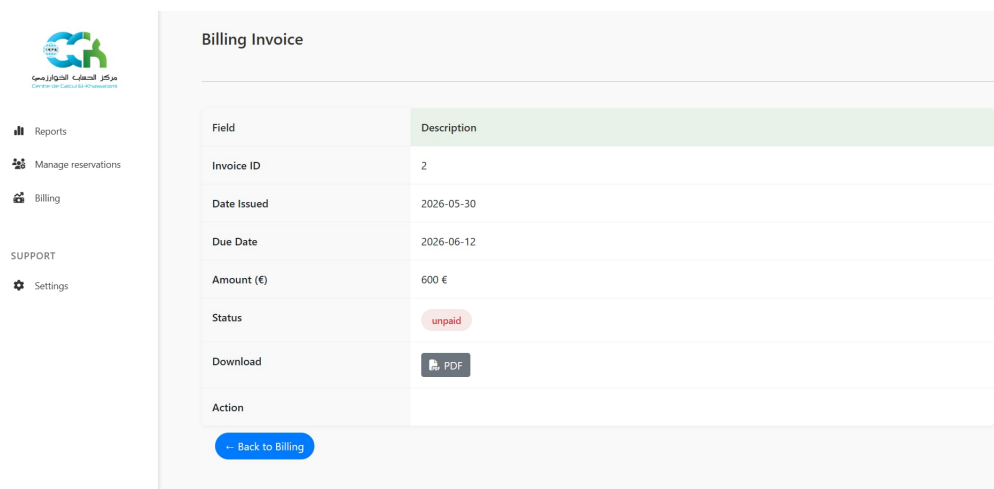


Figure 4.10: Interface: University Bill details

Universities are provided with bill details interface with PDF download option.

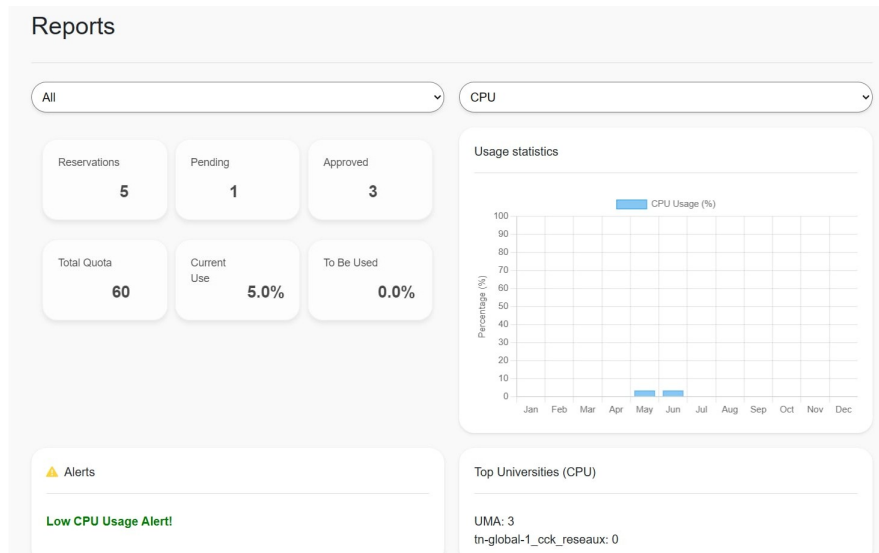


Figure 4.11: Interface: CCK Reports
CCK admins are provided with statistics per resource.

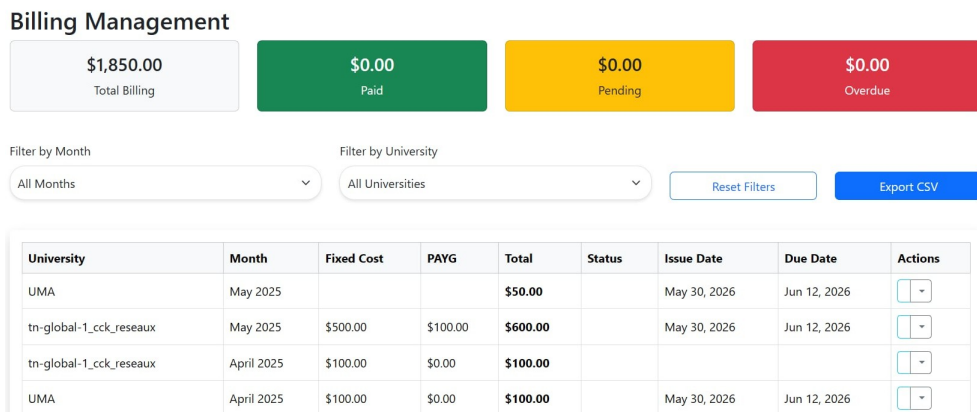


Figure 4.12: Interface: CCK Billing
CCK admins can consult universities' bills details.

Conclusion

In this chapter, we have detailed the steps involved in implementing the features in Release 2, starting with the requirements analysis, then the design, and finally the realization.

CONCLUSION AND PERSPECTIVES

As part of our final year project, we designed and developed a platform for cloud resources reservation and billing. This platform provides a reliable system that will be used by CCK to process operations related to academic cloud resources reservation.

Thanks to our academic training and the new knowledge acquired during our internship, we largely achieved the objectives we initially set. Specifically, we were able to deliver essential functionalities to the platform, including a secure login system for administrators and professors, a dashboard that reports real-time metrics on cloud resource consumption, a reservation module that allows professors to request and manage cloud resources, a billing module that tracks consumption per university and generates invoices based on actual usage, and a reporting feature that provides detailed statistics on resource consumption.

This final year project was very helpful for us because we were able to find new ways of developing and using new technologies. While it was very enriching, it also presented us with challenges, as it was our first time using SCRUM and some of the new technologies. Therefore, we devoted a lot of time to learning and mastering these new tools.

Despite the time constraints and obstacles encountered, we managed to improve our performance and knowledge by moving from design to implementation, putting what we learned into practice, and overcoming all sorts of problems. Ultimately, this experience was very educational and helped us acquire valuable new skills.

There are many ways to improve our project; some of these include expanding the platform with features such as a forecasting module to predict future consumption of resources by category (ex: CPU, storage, RAM, etc.), which would allow universities to better anticipate demand and improve resource planning. The platform could additionally be developed into a mobile version of the platform to allow professors and staff to manage reservations and monitor resources on the go.

Bibliography

- [ASI07] Noraida Haji Ali, Zarina Shukur, and Sufian Idris. “A design of an assessment system for UML class diagram”. In: *2007 International Conference on Computational Science and Its Applications (ICCSA 2007)*. IEEE. 2007, pp. 539–546.
- [ASK05] Ludovic Apvrille, P de Saqui-Sannes, and Ferhat Khendek. “Synthèse d’une conception UML temps-réel à partir de diagrammes de séquences”. In: *Colloque Francophone sur l’ingénierie des protocoles (CFIP’05), Bordeaux, France*. Vol. 112. 2005.
- [BAT14] Gavin Bierman, Martín Abadi, and Mads Torgersen. “Understanding type-script”. In: *ECOOP 2014–Object-Oriented Programming: 28th European Conference, Uppsala, Sweden, July 28–August 1, 2014. Proceedings 28*. Springer. 2014, pp. 257–281.
- [Bd00] Eric BRASSART and MCFIUT informatique d’Amiens. *Le Langage HTML Hyper Texte Markup Language*. 2000.
- [Bha15] Snig Bhaumik. *Bootstrap essentials*. Packt Publishing Ltd, 2015.
- [Boo20] Spring Boot. *Spring Boot*. 2020.
- [CDD19] Jelica Cincovic, Sanja Delcev, and Drazen Draskovic. “Architecture of web applications based on Angular Framework: A Case Study”. In: *methodology* 7.7 (2019), pp. 254–259.
- [Che+23] Brian Chen et al. “VSCuda: LLM based CUDA extension for visual studio code”. In: *Proceedings of the SC’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. 2023, pp. 11–17.
- [Cro08] Douglas Crockford. *JavaScript: gardez le meilleur!* Pearson Education France, 2008.
- [CS16] Alain Collignon and Joachim Schöpfel. “Méthodologie de gestion agile d’un projet. Scrum–les principes de base”. In: *I2D-Information, donnees documents* 53.2 (2016), pp. 12–15.
- [Fau+19] Reza Fauzan et al. “Use case diagram similarity measurement: A new approach”. In: *2019 12th International Conference on Information & Communication Technology and System (ICTS)*. IEEE. 2019, pp. 3–7.
- [Gor02] Davor Gornik. “UML data modeling profile”. In: *White Paper, Rational Software* (2002).
- [KF] G Kurzawa and G Farges. “LES CLES POUR DEMARRER AVEC L’AGILITE”. In: ().

- [Kor+22] Pratik Prakash Kore et al. “API Testing Using Postman Tool”. In: *International Journal for Research in Applied Science and Engineering Technology* 10.12 (2022), pp. 841–43.
- [LEB03] Rémi LEBLOND. *Vers une architecture n-tiers*. 2003.
- [Mey06] Eric A Meyer. *CSS: The Definitive Guide: The Definitive Guide.* ” O’Reilly Media, Inc.”, 2006.
- [MG00] Pierre-Alain Muller and Nathalie Gaertner. *Modélisation objet avec UML*. Vol. 514. Eyrolles Paris, 2000.
- [NBG24] Oksana Nikiforova, Kristaps Babris, and Aytaj Guliyeva. “Definition of a Set of Use Case Patterns for Application Systems: A Prototype-Supported Development Approach”. In: *Applied Computer Systems* 29.1 (2024), pp. 59–67.
- [Oma] Omara. *Spring*. URL: [https : / / ocamil . com / index . php / spring / generalites](https://ocamil.com/index.php/spring/generalites).
- [Sac16] Sakshi Sachdeva. “Scrum methodology”. In: *Int. J. Eng. Comput. Sci* 5.16792 (2016), pp. 16792–16800.
- [Sar21] Debjani Sarkar. “Introducing LaTeX to the Academic Researcher: Engineering Writing with a Difference (RESUBMISSION)”. In: *2021 First-Year Engineering Experience*. 2021.
- [Sin+19] Mandeep Singh et al. “Implementation of database using python flask framework”. In: *International Journal of Engineering and Computer Science* 8.12 (2019), pp. 24890–24893.
- [Sta22] Fabio Staiano. *Designing and Prototyping Interfaces with Figma: Learn essential UX/UI design principles by creating interactive prototypes for mobile, tablet, and desktop*. Packt Publishing Ltd, 2022.
- [Wor13] MySQL Workbench. “MySQL Workbench”. In: *Obtenido de* (2013).

Abstract

This report exposes the work done in the company EL KHAWARIZMI Computer Center as part of the final year project to obtain a bachelor's degree in Computer Science, specializing in computer science and multimedia for the academic year 2025-2026.

In this project, our primary goal is to streamline the reservation system for professors, centralize cloud resource usage data and automatically generate billing reports for universities.

This system aims to significantly enhance efficiency, security and transparency of resource management within educational institutions.

Keywords: Angular, Flask, Spring Boot, REST API, Cloud.

Résumé

Ce rapport présente le travail réalisé au sein de l'entreprise Centre de Calcul EL KHAWARIZMI dans le cadre du projet de fin d'études pour l'obtention de la licence en informatique, spécialité informatique et multimédia, pour l'année universitaire 2025-2026.

Ce projet vise principalement à rationaliser le système de réservation des professeurs, à centraliser les données d'utilisation des ressources cloud et à générer automatiquement des rapports de facturation pour les universités.

Ce système vise à améliorer considérablement l'efficacité, la sécurité et la transparence de la gestion des ressources au sein des établissements d'enseignement.

Mots clefs : Angular, Flask, Spring Boot, API REST, Cloud.

